KTH

DEGREE PROJECT IN COMPUTER SCIENCE AND ENGINEERING
SECOND CYCLE, 30 CREDITS

Stockholm, Sweden 2026

Joint Radar Emitter and Mode Clustering Using a Transformer Encoder Architecture

Markus Swegmark

Supervisor: TBD
Examiner: TBD
Host: TBD

KTH Royal Institute of Technology
School of Electrical Engineering and Computer Science
Master's Programme in Machine Learning

TRITA-EECS-EX-XXXX:XXX

Abstract

This thesis investigates joint clustering of radar emitters and their operating modes from intercepted pulse descriptor word (PDW) streams using a transformer encoder architecture.

Keywords: transformer, deep clustering, electronic intelligence, radar emitter identification, mode recognition, supervised contrastive learning.

Sammanfattning

Detta examensarbete undersöker gemensam klustring av radarsändare och deras operativa moder från avlyssnade pulsbeskrivande ord (PDW) med hjälp av en transformerbaserad encoderarkitektur.

Nyckelord: transformer, djup klustring, signalspaning, radarsändaridentifiering, modigenkänning, självövervakad inlärning.

Acknowledgments

I would like to thank my supervisor and examiner at KTH, and the host organization for their support throughout this thesis project.

Stockholm, August 2026
Markus Swegmark

Contents

List of Figures

List of Tables

List of Acronyms

ACC clustering accuracy

AMI adjusted mutual information

AOA angle of arrival

ARI adjusted Rand index

BERT Bidirectional Encoder Representations from Transformers

DBSCAN Density-based spatial clustering of applications with noise

DEC deep embedded clustering

DL deep learning

ELINT electronic intelligence

ESM electronic support measures

EW electronic warfare

FFN feed-forward network

GMM Gaussian mixture model

HDBSCAN Hierarchical density-based spatial clustering of applications with noise

MHA multi-head attention

ML machine learning

MLP multilayer perceptron

PDW pulse descriptor word

PRF pulse repetition frequency

PRI pulse repetition interval

RADAR radio detection and ranging

RF radio frequency

RoPE rotary positional encoding

TOA time of arrival

V-measure harmonic mean of homogeneity and completeness

CHAPTER 1

Introduction

1.1 Background and Motivation

Recent large-scale conflicts illustrate how contested the electromagnetic spectrum has become. Commanders rely on timely, passive sensing to build situational awareness, protect platforms, and coordinate fires without revealing their own emissions. Within EW, which aims to exploit, protect, and maneuver in the spectrum, ESM systems listen passively and supply the measurements that feed higher-level ELINT tasks such as detecting radars, grouping pulses by emitter, and inferring behavior [1, 2].

Modern radars are increasingly agile: they vary carrier frequency, pulse width, amplitude, and PRI across bursts to improve performance and survivability. That agility blurs simple fingerprints and stresses classical rule-based parsers tuned to stable pulse trains. The same physical emitter also switches *modes*—recurrent control policies that map an operational objective (for example, volume search versus track) to a characteristic regime in the pulse train. Mode awareness is therefore operationally informative: it constrains how the waveform may evolve, signals intent at a coarse level, and can help disambiguate emitters when type-level signatures overlap.

At the same time, ML and DL have become the default toolkit for sequence modeling and representation learning in other domains. Radar pulse analysis naturally presents as a stream of low-level measurements—often summarized as PDWs—where long-range dependence and mixture structure (multiple emitters superposed in time) favor learned encoders over purely hand-crafted feature pipelines, and where the available supervision is itself asymmetric: emitter identifiers in curated training data are typically defined per pulse train rather than across an open global population, while operating modes are drawn from a much smaller, shared catalogue.

This thesis studies whether a transformer-style encoder, trained with objectives suited to clustering, can produce embeddings in which both distinct emitters and their operating modes emerge as separable structure. The remainder of the introduction states the problem precisely, lists the research questions, and outlines scope, contributions, and ethics.

1.2 Problem Statement

Passive ESM produces a time-ordered stream of PDWs in which pulses from multiple emitters are interleaved and each emitter may change operating mode over time (Section 1.1). Operators ultimately require both emitter grouping and operating-mode information from the same measurement stream, and classical pipelines therefore chain deinterleaving with separate mode heuristics or specialised supervised classifiers; under agile waveforms and overlapping trains, that separation scales poorly and does not guarantee a single representation whose geometry exposes emitter- and mode-level structure simultaneously. This thesis addresses the joint problem in a supervised setting: training pulse trains are generated by a controlled scenario simulator (Section 4.2) in which every PDW carries two ground-truth labels with asymmetric scope — a per-pulse emitter identity that is unique only *within* the pulse train that contains the pulse, and an operating mode drawn from a finite catalogue that is shared across all pulse trains. The task is therefore to learn from this label structure a single representation in which (i) the train-local emitter labels can be used to cluster pulses by physical emitter at inference time, including on previously unseen trains whose number of emitters is not known in advance, and (ii) the global mode labels can be used to cluster pulses by operating mode, including on modulation types not present in the training catalogue, while remaining robust to mixture effects, noise, and imperfect preprocessing.

1.3 Research Questions

The thesis addresses the following research questions:

RQ1 Under practical sequence-length limits of a transformer encoder, can the model support effective deinterleaving and recover operating modes via clustering?

RQ2 Does a joint objective that couples deinterleaving with mode-aware clustering yield better clustering performance or representations than a comparable single-objective transformer baseline focused on deinterleaving alone?

1.4 Objectives and Scope

1.4.1 Objectives

The work pursues objectives that operationalize the research questions in Section 1.3. First, we specify and implement a transformer-style encoder that ingests bounded windows of PDWs from interleaved pulse streams and maps them to a dual embedding (Section 4.1), together with a post-hoc clustering stage applied independently to the emitter and the mode branches. We then study how performance depends on practical limits on context length, mixture density, and noise, reporting agreement with the ground-truth simulator partitions using standard scores such as AMI and ARI (Chapter 5). Second, we formulate at least one joint training objective that couples a within-train supervised contrastive term for emitter identity with a batch-wide supervised contrastive term over the global mode labels, and compare it against a matched baseline trained primarily for deinterleaving or representation quality without that coupling, holding architecture capacity and data conditions as constant as feasible (Chapters 4 and 5).

1.4.2 Delimitations

The scope is deliberately narrow. We operate at the level of PDWs or equivalent tabular pulse features supplied to the model; pulse detection, RF conditioning, antenna-level processing, and geometry-driven fusion across platforms are out of scope. All training and evaluation data are generated by a proprietary scenario simulator (Section 4.2): scenarios are run and the resulting pulse data are collected, including PDWs and the corresponding per-pulse emitter and mode labels. No operational ELINT recordings are used, so the conclusions are not validated against real-world distribution shift, and receiver-side degradations such as dropped pulses and spurious detections are simulator-generated rather than measured artifacts.

The focus is offline or batch analysis of recorded streams rather than hard real-time implementation on embedded hardware, and we do not address operational emitter naming against classified libraries. Because the self-attention mechanism of the transformer scales quadratically with sequence length, recorded scenarios are split into bounded windows that are processed individually (Section 4.2.2); conclusions therefore concern representation quality and clustering agreement on such windows rather than end-to-end tracking performance over arbitrarily long horizons. Extensions to online operation and to library-based identification are discussed in Sections 6.3 and 6.4 but are not implemented.

Finally, all evaluation scenarios draw their operating modes from the same simulator catalogue used during training (Section 5.2); although the clustering-based mode output does not presuppose a closed catalogue, generalization to modulation types absent from training is not evaluated, and no claim is made about novel waveforms.

1.5 Contributions

The main contributions of this thesis are:

- Placeholder contribution.
- Placeholder contribution.

1.6 Ethical and Societal Considerations

Methods for passive ESM and ELINT sit squarely in dual-use territory: the same representations that support defensive situational awareness, spectrum management, and safety-of-navigation applications can, in principle, support targeting or surveillance against specific platforms if deployed without appropriate policy, oversight, and legal constraints. This thesis develops ML techniques on abstract pulse-level features and does not address deployment, rules of engagement, or classification of live operational data; nevertheless, the capability to group emitters and recognise operating modes from PDW streams should be understood as a general-purpose building block whose societal consequences depend on the institutional context in which it is used.

From a data-ethics perspective, the experimental work relies on controlled or synthetic PDW streams rather than collections derived from live emitters, which limits privacy and classification sensitivity concerns at the cost of weaker guarantees about behavior under classified or vendor-specific waveforms. When such methods are extended toward operational data, stewards must enforce access control, purpose limitation, and retention policies consistent with national regulation and organizational policy, and they should document provenance and limitations so that operators do not over-trust cluster labels that lack ground truth.

Training modern DL encoders incurs non-negligible energy use for computation and cooling; relative to lightweight classical deinterleaving heuristics, transformer-scale models increase the carbon footprint of experimentation and of any future always-on training loops. Mitigation follows standard practice: reuse checkpoints, avoid redundant hyperparameter sweeps, prefer smaller models when they meet accuracy targets, and report approximate training cost where it informs reproducibility.

At a societal level, automation of emitter and mode analysis may concentrate analytic capacity in well-resourced actors, widen asymmetries in electromagnetic situational awareness, and reduce the visibility of human expert judgment if clusters are treated as authoritative without audit trails. Responsible research therefore favors transparent evaluation protocols, clear statements of failure modes under noise and mixture, and conservative claims about operational readiness.

1.7 Thesis Outline

The remainder of the thesis is organized as follows. Chapter 2 supplies textbook-style background on pulsed radar, PDWs, emitters and operating modes, the ELINT processing chain, and the ML and transformer concepts needed to read the method. Chapter 3 turns to the literature, contrasting classical emitter identification with deep supervised radar models, deep clustering objectives, sequence transformers, and work on joint or hierarchical cluster structure; each subsection closes with the limitation that motivates the present approach. Chapter 4 states the formal problem, documents data representation and preprocessing (Section 4.2), specifies the dual-branch encoder and losses (Sections 4.3 and 4.5), describes training and metrics, and lists baselines. Chapter 5 reports hardware and software setup, the synthetic scenario corpus (Section 5.2), hyperparameters, quantitative scores, ablations, and qualitative analyses. Chapter 6 interprets findings, positions them against prior art, states limitations—including simulator-only evidence and operational gaps—and discusses implications.

Chapter 7 summarizes answers to the research questions and outlines future work.

CHAPTER 2

Background

2.1 Radar Systems and Passive Reception

2.1.1 Electromagnetic Pulses for Navigation, Surveillance, and Control

The term RADAR stands for **Ra**dio **D**etection **a**nd **R**anging. At its core, a radar system operates by sending out electromagnetic pulses and then listening for the echoes that reflect back from objects in the environment. This basic principle supports a wide range of applications, including air traffic control, weather monitoring, altitude measurement, navigation of ships and vehicles, target tracking, and monitoring of the electromagnetic spectrum alongside commercial communication services [2].

What unifies all radar systems is a practical goal: they emit a carefully controlled burst of electromagnetic energy, allow it to scatter from surrounding objects, and then extract information about the location and motion of those objects from the returning signals. Electromagnetic waves are generated whenever electric charges accelerate, such as when an alternating current passes through an antenna. These waves move outward from the antenna at the speed of light in air. In radar, each outgoing pulse can be thought of as a brief and well-defined packet of energy. By recording the time it takes for this energy to bounce off an object and return, and knowing the speed at which it traveled, the radar can directly calculate how far away the object is [2].

2.1.2 Time-of-Flight Ranging and Ambiguity Fundamentals

At its core, radar sensing uses the physics of wave propagation to measure distance. A transmitter emits a brief electromagnetic pulse, which travels at speed c (the speed of light in air), reflects from an object, and returns as an echo. By measuring the elapsed time τ between emission and reception—known as the round-trip delay—the system can infer the distance to the object.

In the canonical configuration, a single (monostatic) radar platform handles both transmission and reception, with the wave traversing to the target and back. The fundamental relation connecting time delay to range follows directly from the definition of speed:

$$R = \frac{c\tau}{2}, \tag{2.1}$$

where R is the target range and c is the wave speed [2]. The factor of 2 accounts for the outbound and return paths.

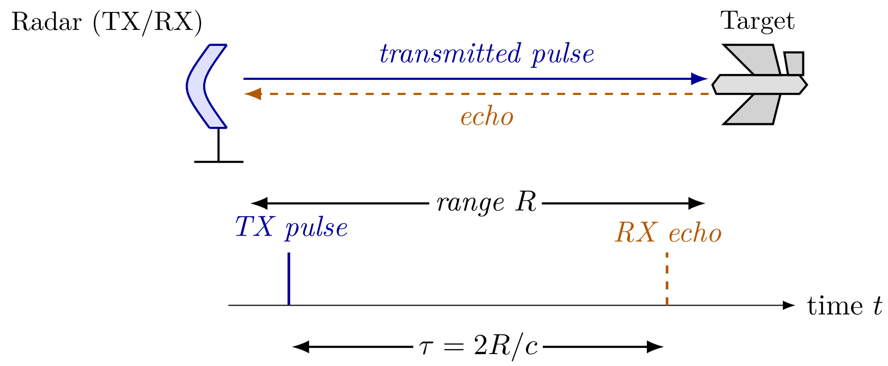

Figure 2.1. Monostatic radar geometry and round-trip timing. A co-located transmit/receive (TX/RX) aperture on the left emits a pulse that travels to a target at range R, scatters, and returns to the same aperture as an echo. The lower timeline shows the corresponding events: the outgoing TX pulse at $t = 0$ and the received echo at $t = \tau$, separated by the round-trip delay $\tau = 2R/c$ from Equation (2.1).

A similar approach is found in animal echolocation: for instance, a bat produces a short sound pulse and estimates the distance to an obstacle or prey by interpreting the delay before it hears the echo, substituting sound waves for electromagnetic ones [2].

While monostatic setups are common, many practical radar and electronic support systems employ multiple spatially separated receive antennas, or distributed arrays, forming so-called bistatic or multistatic configurations. By correlating the signals arriving at different locations, these systems can measure angles of arrival and other spatial properties in addition to range, thus supporting direction finding and improved localization.

Practical radar systems emit periodic pulses, each separated by a pulse repetition interval (PRI) or, equivalently, a pulse repetition frequency (PRF). A core challenge appears when echoes from earlier pulses arrive after a new pulse is sent. This leads to *range ambiguity*, where it is unclear which transmitted pulse a received echo should be attributed to. The largest range without ambiguity—the *unambiguous range*—is set by the time between pulses:

$$R_\mathrm{u} = \frac{c}{2\,\mathrm{PRF}} = \frac{c\,\mathrm{PRI}}{2}. \tag{2.2}$$

Signal processing techniques such as matched filtering and coherent integration increase detection performance within this range, but no algorithm can resolve targets unambiguously beyond R_u as limited by the physics of pulse timing [2].

2.1.3 Transmitting radars versus receive-only systems

Any receiver within range can detect intentional electromagnetic transmissions, and repeated radar pulses make timing and direction information available not only to the radar user, but to anyone listening in [1]. This exposes a fundamental trade-off: transmitting enables the measurement of objects at a distance, but simultaneously reveals the radar's presence, position, and activity to others in the environment.

Traditional (transmitting) radar systems measure the delay between their own outgoing pulses and the returning echoes, allowing direct range estimation [2]. In contrast, passive systems such as ESM and ELINT do not transmit their own signals. Instead, they monitor emissions from other sources and attempt to infer information about those emitters and their behavior [1]. Because they do not transmit, passive receivers maintain a lower profile and avoid revealing themselves by radiation—a key advantage in situations where stealth is important.

Lacking direct control over the transmitted waveform, passive receivers rely on whatever pulse measurements they can extract after detection: carrier frequency, time of arrival, pulse width, amplitude, and angle of arrival if available. This necessity motivates the PDW abstraction described next. Section 2.2 shows how PDWs fit into the overall signal processing chain.

2.1.4 From the raw signal to a pulse sequence and descriptor words

Every radar or receiver starts by collecting a raw electrical signal—essentially a voltage that varies in time as electromagnetic waves are picked up by the antenna. This raw signal can be viewed as a digital recording of the environment's radio activity, capturing everything present, including both the pulses of interest and background noise.

To extract information, the system looks for brief increases in signal energy that stand out above the noise. These increases correspond to pulses—short bursts of transmitted energy. Locating a pulse means finding where in time the signal rises above a set threshold. After locating pulses, the receiver estimates several physical properties for each one: when it arrived (its time of arrival), how long it lasted (its pulse width), how strong it was (amplitude or energy), and what frequency it had. If the receiver has directional antennas, it can also estimate the angle from which the pulse arrived.

After initial detection and measurement, each pulse is summarized into a standardized set of numbers called a *PDW* (pulse descriptor word). This record captures the essential measured properties of a pulse, such as:

$$\boldsymbol{x}_n = \Big(f_n,\, \mathrm{PW}_n,\, t_n,\, A_n,\, \theta_n,\, \ldots\Big)^\top \in \mathbb{R}^d, \tag{2.3}$$

where f_n is the measured carrier frequency, PW_n is pulse width, t_n is time of arrival (often the leading edge), A_n is amplitude, and θ_n is angle of arrival if available. The ellipsis allows for extra recorded properties, like elevation angle of arrival, but the main idea is that each column represents a specific physical measurement, and the format is fixed once chosen. Figure 2.2 visualizes this five-parameter template.

A sequence of such pulses, indexed by order of arrival $(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots)$, forms what is called a *pulse train*. For pulses that come from a single transmitter operating in a stable mode, the intervals between arrivals ($\tau_n = t_n - t_{n-1}$, for $n \geq 2$) are usually regular or follow a set schedule. This interval is called the pulse repetition interval (PRI); its

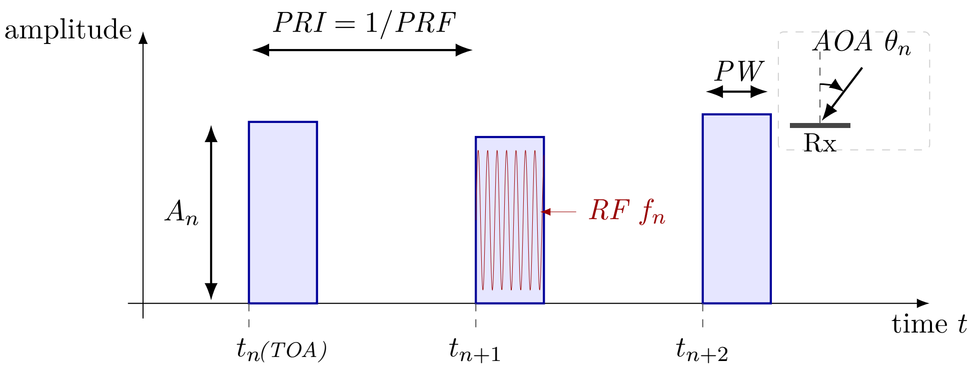

Figure 2.2. Schematic of the parameters captured by a PDW for a short pulse train. Each pulse contributes a TOA (the leading edge t_n on the time axis), a pulse width (PW), a peak amplitude A_n, and a carrier RF f_n, sketched as the fast oscillation inside the middle pulse; successive pulses also define a PRI (equivalently a PRF = 1/PRI). The inset shows the AOA θ_n measured at the receive aperture relative to a boresight reference. A typical PDW stacks these per-pulse measurements into a vector $\boldsymbol{x}_n = (f_n, \mathrm{PW}_n, t_n, A_n, \theta_n)$ that is the usual input to downstream emitter and mode reasoning [1].

inverse, the pulse repetition frequency (PRF), describes how often pulses are sent:

$$\tau_n = t_n - t_{n-1}, \qquad n \geq 2. \tag{2.4}$$

While simple systems use a fixed interval, more complex transmitters may change their intervals in a predictable or agile way, and this information can be included in the record for each pulse.

Each measured descriptor is thus a function of both the true (but unknown) properties of the transmitting source and the effects of noise, measurement error, and possible missing data. Very weak pulses may be missed, and noise or spurious events may be incorrectly identified as pulses.

Finally, rather than carrying forward the full raw signal, the system represents each pulse only by its measured properties in the form of a time-ordered sequence of PDWs. This sequence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T)$—sometimes called the pulse train—is the main input to later processing and learning algorithms. Each $\boldsymbol{x}_n$ is a compact summary of one pulse, and the order reflects the timeline of arrivals, replacing the need for direct inspection of the original sampled voltage trace.

2.1.5 Physical sources and transmit schedules

In passive ESM and ELINT, a *radar emitter* means a physical transmitter whose pulses line up in time and frequency strongly enough to be treated as one source [1]. An *operating mode* is the instantaneous transmit recipe: carrier plan, pulse width, repetition pattern, beam motion, and any programmed switching rules among them [1, 2]. The same hardware cycles among modes as tasks change, so a timeline of PDWs mixes mode segments even when the physical box is fixed. Different platforms can also implement the same functional mode with different numerical parameters, for example two trackers with distinct nominal PRFs [1].

Illustrative mode families include wide-area search, acquisition and track, fire-control illumination, weather sensing, and terminal guidance support [2]. Those roles push different combinations of PRI, dwell, agility, and scan law, which is why operators care both about identity and about what the transmitter is doing at a given time [1, 2].

The relationship between emitters and modes is many-to-many: one emitter runs many modes over a mission, and different emitters can realize similar modes with different parameter sets [1]. The rest of this chapter walks through the passive processing chain (Section 2.2) and then supplies the learning background used in the method.

2.2 Signal Processing Pipeline for Passive ELINT

In EW and ESM applications, measured radar energy passes through a staged *signal processing pipeline* before it reaches higher-level ELINT reasoning. The exact implementation varies with platform, band, and mission software, but the logical chain is broadly stable across textbooks and system descriptions.

2.2.1 Front-end reception, detection, and pulse-parameter estimation

At the *front end*, antennas and RF conditioning capture emissions over one or more instantaneous bandwidths, often after wideband digitization or channelized sub-band processing. A *detection* stage then marks pulse-like events against a noise and clutter background, producing a sparse list of candidate pulse times (and sometimes coarse frequency hints).

Given detections, *parameter estimation* attaches per-pulse measurements such as carrier frequency, pulse width, time of arrival, direction of arrival when available, and amplitude or signal-to-noise proxies, this becomes our PDWs after possibly attatching AoA after triangulation. These estimates are noisy, can be missing for weak emitters, and may include false alarms that are not associated with any physical radar.

2.2.2 Deinterleaving and pulse-train formation

Sorting and *deinterleaving* group pulses into coherent pulse trains by exploiting periodic structure in inter-pulse timing and consistent parameter tracks across time. The output of this stage is commonly packaged as a stream of PDWs (or closely related pulse descriptor records), which form the usual interface to downstream tasks such as emitter identification and mode analysis (Section 2.1.4).

Operationally, deinterleaving developed from deterministic temporal rules toward multi-parameter clustering and, more recently, toward learned sequence models [3]. In comparatively sparse environments with simple pulse trains, PRI inferred from TOA differences often behaved like a stable timing signature, so early algorithms searched for repetition by accumulating histograms of inter-pulse intervals. Cumulative and sequential difference histograms (CDIF and SDIF) are representative: they "vote" for candidate PRI values through peaks in the difference domain and remain standard pedagogical references [1, 3]. Their accuracy rests on approximately regular timing; dense mixing, harmonic ambiguity, false peaks from cross-emitter differences, and missing pulses that break sequential differencing all erode reliability [3].

Once PRI agility became widespread, many systems augmented timing with unsupervised structure in joint spaces of RF, pulse width, AOA, and related measurements, using methods such as k-means or density-based clustering [3, 4]. That shift treats deinterleaving as geometric separability in hand-crafted feature space, but performance still depends on scaling, metric choices, and hyperparameters (for example cluster counts or neighborhood radii) that are difficult to set when emitters are unknown a priori [3].

Modern emitters compound the difficulty through overlapping pulse trains, rapid mode changes, and low-observable emissions that increase loss and measurement jitter [3]. The PDW representation in Section 2.1.4 and the learning-based tools in later sections of this chapter target reasoning under those conditions.

In classical ELINT architectures, specifications therefore treat train formation as an early, largely discrete step: deinterleaving is completed first so that downstream logic receives one dominant hypothesis per physical emission path, or a short ranked list, rather than a time-interleaved mixture [1].

2.2.3 Emitter reporting and equipment identification

Emitter reports are assembled after train formation, summarizing each train with aggregated PDW statistics, tentative equipment identities from lookup tables, and temporal extent. Library-driven identification and threat correlation consume those reports together with operator context.

2.2.4 Operating-mode analysis and waveform libraries

Mode classification is then commonly framed as matching the train against a *mode library* of nominal RF bands, PRI/PRF families, stagger laws, and scan programs, so that declared modes remain anchored to catalogued waveform templates. When labels are incomplete, unsupervised or weakly supervised grouping over pulse trains plays the same operational role as *mode clustering* at the library boundary: it proposes coherent behavior segments that analysts later align to doctrine or intelligence products.

2.2.5 Emitter geolocation, fusion, and downstream displays

Positional estimation—for example AOA intersections from direction-finding networks, time-difference-of-arrival fixes along baselines, or fused tracks across collectors—consumes the same deinterleaved associations, which isolates geometry and timing errors from the sorting stage. Later stages may maintain libraries, threat lists, and operator displays, but for the learning problem in this thesis the critical contract is that the model consumes the deinterleaved PDW stream produced here.

2.3 Machine Learning Fundamentals

At its most general, ML replaces hand-coded decision rules with parametric models whose parameters are fitted to data so that a chosen performance criterion is optimized on observed examples of the task. A model is a function $f_{\boldsymbol{\theta}} \colon \mathcal{X} \to \mathcal{Y}$ from an input space $\mathcal{X}$ to a task-specific output space $\mathcal{Y}$, and learning consists of searching over parameter vectors $\boldsymbol{\theta}$ until $f_{\boldsymbol{\theta}}$ is consistent with the regularities present in the training data. What changes between problems is the nature of the supervision available, the structure of $\mathcal{Y}$, and the loss that scores predictions against that supervision.

A common formal frame is *empirical risk minimization.* Given a dataset $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$ drawn from an unknown distribution and a pointwise loss ℓ, the learner solves

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \ \frac{1}{N} \sum_{i=1}^{N} \ell\Big(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i),\, y_i\Big) + \Omega(\boldsymbol{\theta}), \tag{2.5}$$

where Ω is an optional regularizer that trades fidelity to the training data against complexity of the fitted model. Generalization to fresh inputs is then assessed on held-out data, and the entire training procedure—data, model class, loss, optimizer, regularization—is judged by its effect on that out-of-sample behavior rather than on the training fit alone.

2.3.1 Learning paradigms

ML problems are conventionally separated by the kind of supervision available at training time. In *supervised* learning, each $\boldsymbol{x}_i$ is paired with a label y_i, categorical for classification or real-valued for regression, and ℓ penalizes disagreement between the prediction and that label. In *unsupervised* learning, labels are absent and the goal is to expose structure in the input distribution itself, for example through density estimation, dimensionality reduction, or clustering; Section 2.4 treats clustering specifically because it is the central downstream consumer of the embeddings used in this thesis. *Self-supervised* learning sits between the two: it constructs supervisory targets automatically from the data through a designed pretext task—predicting masked parts of an input, contrasting augmented views, or ordering shuffled tokens—so that an encoder can be trained without manual annotation. The remainder of this thesis operates in the supervised regime: training pulse trains carry per-pulse emitter and operating-mode labels (Sections 4.1 and 4.2), and the training objective uses both label streams directly; the unsupervised and self-supervised paradigms are summarised here only because the clustering tools in Section 2.4 and the contrastive representation-learning machinery in Section 2.5 are inherited from those traditions.

2.3.2 Representation learning

Much of modern DL practice is best understood as *representation learning*: a parametric encoder maps high-dimensional observations into a lower-dimensional vector space whose geometry is shaped by the training objective. Downstream tasks—classification, clustering, retrieval, or sequence prediction—then operate on the resulting *embeddings* $\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x}) \in \mathbb{R}^d$ rather than on raw sensor coordinates. The advantage of routing tasks through a learned representation is that invariances, similarities, and task-relevant factors of variation can be encoded by $f_{\boldsymbol{\theta}}$ once, after which simple linear or geometric operations on $\boldsymbol{z}$ suffice for many downstream questions. Sections 2.5 and 2.6 expand on this view; the method chapter (Chapter 4) instantiates it for PDW streams, training an encoder whose embeddings are intended to expose emitter and mode structure.

2.3.3 Applications in radar deinterleaving and mode classification

Radar signal deinterleaving and operating-mode classification illustrate the practical pull of these ideas. For decades, both tasks were addressed by deterministic temporal rules and hand-crafted statistics over PDW parameters, as outlined in Section 2.2. As emitters became more agile and electromagnetic environments more crowded, the field migrated first to *shallow* ML in the form of multi-parameter clustering—k-means and density-based methods over RF, AOA, and pulse width—and more recently to *deep* sequence models that learn pulse-level features directly from data [3]. Recurrent and attention-based architectures now treat deinterleaving as sequence labeling in which the per-pulse output identifies an emitter, while transformer encoders in particular are well suited to capturing the non-contiguous, multi-scale relationships induced by jittered, staggered, or grouped PRIs [3, 5].

Mode classification follows a parallel trajectory. Classical ESM systems match deinterleaved pulse trains against catalogued waveform templates in a mode library [1], while learning-based approaches reframe the same task either as supervised classification over known mode labels or as unsupervised grouping over pulse trains when labels are scarce or incomplete [3]. This thesis occupies an intermediate position: training uses explicit per-pulse mode labels from a global catalogue, but the training signal is a supervised contrastive loss that shapes embedding geometry without a closed-catalogue softmax, so that previously unseen modulation types can surface as additional clusters at inference; the emitter task uses the same kind of supervised contrastive signal but with labels whose scope is local to each pulse train, and inference also runs through clustering. The clustering tools in Section 2.4 and the contrastive representation-learning machinery in Sections 2.5 and 2.6 provide the technical foundation that the method chapter builds on.

2.4 Clustering Methods

Clustering partitions a finite set of points $\{\boldsymbol{z}_i\}_{i=1}^N \subset \mathbb{R}^d$ into groups so that points inside a group are more similar to one another than to points in other groups. In this thesis, the points are pulse embeddings and the groups are either emitters within a window or operating modes; the clustering operator is applied only at inference, after the encoder has been trained (Section 4.3). Classical algorithms differ mainly in how similarity is defined, whether the number of clusters must be known in advance, and how they treat outliers.

Centroid-based methods such as k-means assign every point to the nearest of k means and update those means until the partition stabilizes. They are fast and well understood, but they assume roughly spherical, equally scaled clusters, require a prescribed k, and force every point into some cluster. Mixture models such as a GMM soften the assignment into soft responsibilities under a parametric density, yet they still typically fix the number of components and remain sensitive to initialization and to elongated or overlapping structure. When the number of concurrent emitters or modes is unknown and the embedding space may contain noise or irregular densities, density-based methods are a more natural fit.

2.4.1 DBSCAN

DBSCAN recovers clusters from local density rather than from a fixed catalogue of centres [4]. Two hyperparameters define the density criterion: a neighbourhood radius $\varepsilon > 0$ and a minimum neighbourhood size minPts $\in \mathbb{N}$. A point is a *core* point if at least minPts points (including itself) lie inside its Euclidean ball of radius ε; points that are not cores but fall inside some core's ball are *border* points; the remainder are labelled *noise*. Clusters grow by connecting core points that mutually fall in each other's ε-neighbourhoods and attaching their border points, so the number of clusters is an output of the run rather than an input.

This behaviour matches the inference setting of Equation (4.14): within each window the number of active emitters (and, separately, of modes) is not known a priori, and sparse outliers can be left unassigned instead of distorting the partition. The main practical cost is sensitivity to ε: a single global radius assumes that all clusters share a comparable density scale. Hierarchical extensions such as HDBSCAN replace the fixed radius with a hierarchy of density levels and extract a flat partition by cluster stability [6, 7], at the expense of additional tuning choices. The method chapter uses plain DBSCAN with frozen $(\varepsilon, \mathrm{minPts})$ on each branch embedding; alternatives appear only as planned ablations (Section 5.5).

Applied to raw high-dimensional features, density thresholds degrade because neighbourhoods become less informative as d grows. Deep clustering therefore first learns a lower-dimensional embedding in which the desired partitions are more linearly or density-separable, then runs a classical operator such as k-means or DBSCAN in that space [8]. Section 2.5 reviews the representation-learning side of that pipeline; the present section supplies only the clustering primitives consumed at inference.

2.5 Deep Representation Learning

Many DL pipelines replace hand-crafted features with a parametric map that is trained end-to-end from data. The central object is an *encoder* $f_{\boldsymbol{\theta}} \colon \mathcal{X} \to \mathbb{R}^d$, implemented in practice as a stack of nonlinear layers (for example, convolutions over a grid, recurrence over time, or attention over a sequence) followed by a pooling or readout head that yields a fixed-dimensional embedding $\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x})$. The design question is which training signal makes $\boldsymbol{z}$ useful for the downstream task when labels are scarce or absent.

In supervised learning, $f_{\boldsymbol{\theta}}$ is trained together with a classifier so that $\boldsymbol{z}$ separates classes under a cross-entropy or margin loss. When cluster structure is the target but assignments are unknown, the same encoder can instead be trained with objectives that encourage smoothness, invariance to nuisance transformations, or agreement with a clustering hypothesis in embedding space. *Self-supervised* learning occupies an intermediate regime: the model predicts surrogate targets constructed from the input itself (for example, masked tokens, relative positions, or transformed views), so that $f_{\boldsymbol{\theta}}$ learns semantics without human labels [9].

Contrastive objectives implement instance discrimination by treating two augmentations of the same example as a positive pair and other examples in a minibatch (or a memory bank) as negatives. Let $\boldsymbol{z}_i$ and $\boldsymbol{z}_i^+$ denote normalized embeddings of two views of sample i, let $\mathrm{sim}(\cdot,\cdot)$ denote cosine similarity, and let $\tau > 0$ be a temperature. A standard *InfoNCE*-style loss takes the form

$$\mathcal{L}_{\mathrm{NCE}}(i) = -\log \frac{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^+)/\tau\Big)}{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^+)/\tau\Big) + \sum_{j \neq i} \exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_j^+)/\tau\Big)}, \tag{2.6}$$

which lower-bounds mutual information between representations of different views under suitable generative assumptions [10]. Large-scale implementations such as SimCLR combine strong augmentations with wide embeddings and many negatives drawn from the same minibatch [9]. Equation (2.6) should be read as a template: the positive set can be enlarged, negatives can be drawn from a queue, and the similarity can be implemented with a learned projection head without changing the underlying contrastive principle.

Pretext tasks offer an alternative route: the network solves an auxiliary problem (predicting rotations, solving jigsaw permutations, inpainting masked regions) whose solution is incidental, but the representation $\boldsymbol{z}$ becomes predictive of semantically stable factors. Such objectives can be combined with clustering when the end goal is partition discovery rather than linear probe accuracy on a fixed label set. Section 3.3 surveys deep embedded clustering and prototype-based alternatives that couple encoder training with discrete structure, together with the gap relative to nested emitter–mode organization in ELINT streams.

Sequence encoders based on self-attention, including the transformer blocks reviewed in Section 2.6, instantiate $f_{\boldsymbol{\theta}}$ when $\boldsymbol{x}$ is an ordered set of tokens (for example, a window of PDWs). The Method chapter builds on this stack: a contrastive or clustering-friendly loss shapes the embedding space, while the attention-based encoder models temporal context within each window.

2.6 The Transformer Encoder

The transformer architecture replaces recurrence with a stack of layers built around *self-attention*, so that every position in a sequence can directly attend to every other position in parallel [11]. One understanding of this is a weighted fully connected graph. Encoder-only stacks, as in BERT [12], map an input sequence to contextualised token representations of the same length and are a natural choice when the downstream task is representation learning or clustering over tokens rather than autoregressive generation.

Scaled dot-product attention. Attention maps three matrices of *queries* $\boldsymbol{Q}$, *keys* $\boldsymbol{K}$, and *values* $\boldsymbol{V}$ to an output matrix of the same leading dimension as $\boldsymbol{Q}$. Each row of $\boldsymbol{Q}$ scores all rows of $\boldsymbol{K}$ through inner products; the scores are scaled, normalised with a row-wise softmax, and used to form a convex combination of the rows of $\boldsymbol{V}$. With d_k the dimension of each query and key vector (column width of $\boldsymbol{Q}$ and $\boldsymbol{K}$ after the usual transpose convention), the mapping is

$$\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\top}}{\sqrt{d_k}}\right)\boldsymbol{V}. \tag{2.7}$$

The scaling by $\sqrt{d_k}$ keeps the inner products from growing too large in magnitude as d_k increases, which would otherwise drive the softmax into near one-hot rows and yield vanishing gradients [11]. Equation (2.7) is reused later as the core attention primitive inside the encoder.

Self-attention and MHA. In *self-attention*, $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are all linear projections of the same input sequence layout, so a token's representation is updated by aggregating information from the full sequence. MHA runs h attention mechanisms in parallel on different learned subspaces, concatenates the head outputs, and applies one more linear map to restore the model width [11]. The heads specialise to different relational patterns while sharing the same residual and normalisation wrapper as a single conceptual operator.

Positional information. Self-attention is permutation-equivariant in its inputs: reordering the rows of $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ in the same way reorders the output rows accordingly, but does not otherwise encode order. Transformer encoders therefore add a positional signal to token embeddings before the first layer, for example fixed sinusoidal features or learned position embeddings added element-wise to each token vector [11, 12]. When the raw inputs already carry absolute or relative timing, as is common for irregularly sampled sequences, practitioners sometimes omit a separate positional encoding and rely on those measurements instead; the method chapter returns to this choice for PDW windows.

Encoder layer. A standard transformer *encoder layer* alternates two sub-layers: MHA followed by a position-wise FFN applied independently at every position. Each sub-layer uses a residual connection and layer normalisation [11]. The FFN is typically a two-layer MLP with a wide inner dimension and a nonlinear activation, acting as the per-token nonlinearity while MHA handles cross-token mixing. Stacking N such layers yields a deep encoder that maps an input sequence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_L)$ to contextualised representations $(\boldsymbol{h}_1^{(N)}, \ldots, \boldsymbol{h}_L^{(N)})$, which downstream heads can pool or read out for sequence-level or token-level predictions.

Computational cost. In the usual dense implementation, each self-attention block materialises an $L \times L$ matrix of token–token scores before the softmax, so time and activation memory for that block scale as $\mathcal{O}(L^2)$ in the sequence length L at fixed embedding width, up to constants from the number of heads and projection ranks [11]. Repeating the block across depth multiplies that cost by the number of layers, which motivates hard caps on L, sparse or low-rank attention variants, or chunked processing when inputs are long; the method chapter states the windowing choice adopted here.

CHAPTER 3

Related Work

3.1 Classical Radar Emitter Identification

Placeholder paragraph.

3.2 Deep Learning for Radar Signal Analysis

Placeholder paragraph.

3.3 Deep Clustering and Joint Representation Learning

Classical clustering assumes that either raw features or a fixed embedding already reveal the desired partition. DEC departs from that split by learning an encoder jointly with a soft assignment to a small set of trainable cluster centroids in embedding space [8]. Training alternates between computing soft assignments from the current encoder and refining the encoder with a Kullback–Leibler objective that sharpens confident assignments toward an auxiliary target distribution, so that the network discovers both features and cluster structure. Follow-on work tightens initialization, adds reconstruction constraints, or refines the target distribution; the common theme is that discrete structure is not fixed in advance but co-adapts with the representation.

SwAV and related *prototype* methods replace dense pairwise contrastive negatives with a prediction problem onto a modest codebook of cluster prototypes, swapping predicted codes between augmented views to avoid representation collapse while still encouraging invariance [13]. They sit between purely self-supervised contrastive pre-training [9] and hard assignment k-means: gradients still flow through discrete-like targets, but the geometry is anchored by learned prototypes rather than by instance indices alone.

Several lines extend deep clustering to *multiple* partitions or hierarchical labels, for example by stacking separate assignment heads, alternating optimization over two clusterings, or enforcing orthogonality constraints between subspaces. The gap relevant to this thesis is narrower: emitter identity and operating mode are not arbitrary independent clusterings—modes nest inside emitters—yet many published pipelines still treat a single flat label space or attach two unrelated classification heads to the same backbone. Work that jointly addresses deinterleaving-style separation, in which emitter identifiers are inherently local to each pulse train and must be recovered by clustering at inference, alongside finer-grained mode grouping that uses global mode labels for supervision but still produces clusters at inference (so that modulation types absent from the training catalogue can surface as additional clusters), all from a shared transformer encoder, remains comparatively sparse; this combination motivates the supervised dual-branch method in Chapter 4.

3.4 Transformers for Time Series and Sets

Placeholder paragraph.

3.5 Joint and Multi-Task Clustering

Placeholder paragraph.

CHAPTER 4

Method

4.1 Problem Formulation

Observed input

The data consist of S pulse trains indexed by $s \in \{1, \ldots, S\}$. Pulse train s is observed as a finite, time-ordered sequence of PDWs

$$\boldsymbol{X}^{(s)} = (\boldsymbol{x}_1^{(s)}, \ldots, \boldsymbol{x}_{N_s}^{(s)}), \qquad \boldsymbol{x}_n^{(s)} \in \mathbb{R}^d,$$

where each $\boldsymbol{x}_n^{(s)}$ contains the measured parameters of one pulse (carrier frequency, amplitude, pulse width, time of arrival, and angles of arrival), and the length N_s varies between pulse trains. The encoder operates on fixed-length windows extracted from these trains as described in Section 4.2.2. When considering one train or window, the superscript is omitted.

Labels and target partitions

Each pulse $n \in \{1, \ldots, N_s\}$ carries an emitter identity $e_n^{(s)} \in \{1, \ldots, K^{(s)}\}$ and an operating mode $m_n^{(s)} \in \{1, \ldots, M\}$. The emitter count $K^{(s)}$ may vary between pulse trains, and each emitter identifier is meaningful only within its train. Mode labels, by contrast, are drawn from a finite catalogue of size M shared across all pulse trains. A train generally contains only $M^{(s)} \leq M$ of these modes. The two labels describe different relations: one emitter may switch between several modes, and the same mode may occur on several emitters.

For pulse train s, the labels induce emitter and mode cells on the pulse-index set,

$$\mathcal{E}_k^{(s)} = \{\, n : e_n^{(s)} = k \,\}, \qquad k \in \{1, \ldots, K^{(s)}\}, \tag{4.1}$$

$$\mathcal{M}_\ell^{(s)} = \{\, n : m_n^{(s)} = \ell \,\}, \qquad \ell \in \{1, \ldots, M\}. \tag{4.2}$$

The emitter cells are non-empty and partition $\{1, \ldots, N_s\}$. The mode cells are also disjoint, but $\mathcal{M}_\ell^{(s)}$ is empty when mode ℓ is absent from the train; the non-empty mode cells form the corresponding partition. Recovering the emitter partition is the deinterleaving task, whereas recovering the mode partition groups pulses by signal behavior irrespective of physical source.

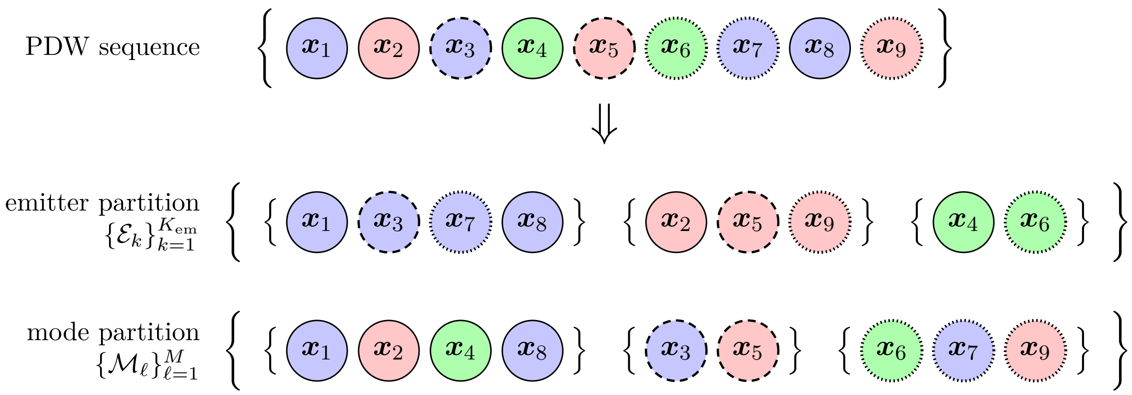

Figure 4.1. Each pulse $\boldsymbol{x}_n$ is assigned both an emitter label e_n (color) and a mode label m_n (border style).

Clustering objective

At inference, only the window $\boldsymbol{X}$ is observed; its labels and the numbers of groups present are unknown. The task is to obtain two per-pulse assignments,

$$\hat{e}_n \in \{1, \ldots, \hat{K}\}, \qquad n \in \{1, \ldots, L\}, \tag{4.3}$$

$$\hat{m}_n \in \{1, \ldots, \hat{M}\}, \qquad n \in \{1, \ldots, L\}, \tag{4.4}$$

where $\hat{K}$ and $\hat{M}$ are inferred by the clustering operator. These assignments induce the estimated partitions

$$\widehat{\mathcal{E}}_k = \{\, n : \hat{e}_n = k \,\}, \quad k \in \{1, \ldots, \hat{K}\}, \qquad \widehat{\mathcal{M}}_\ell = \{\, n : \hat{m}_n = \ell \,\}, \quad \ell \in \{1, \ldots, \hat{M}\}. \tag{4.5}$$

The joint clustering objective is to recover, within each window, the restrictions of both ground-truth partitions in Equations (4.1) and (4.2) to that window, up to independent permutations of their cluster indices. The cluster symbols themselves carry no meaning; agreement depends only on which pulses are grouped together.

Both assignments are produced per window, so their predicted cluster symbols are local to that window. Emitter identities are additionally meaningful only within the pulse train from which the window was drawn. Neither $\hat{K}$ nor $\hat{M}$ is assumed known, and $\hat{M}$ is not constrained by the training catalogue size M; this permits clusters corresponding to modulation types not observed during training.

Learning objective

The proposed method realizes this objective with a shared encoder and two embedding branches, one for emitter identity and one for operating mode. Training uses the labelled pulse trains and two supervised contrastive losses with the same functional form; train-local emitter labels supervise one branch, while globally shared mode labels make the other branch comparable across pulse trains. The losses are specified in

Section 4.5, and the architecture and post-hoc clustering procedure are described in Section 4.3.

4.2 Data Representation and Preprocessing

All training and evaluation data are synthetic because operational ELINT recordings are scarce and sensitive. A scenario simulator generates time-ordered PDW recordings with varying numbers of emitters and operating modes [14]. Each pulse contains the measured parameters and per-pulse labels defined in Section 4.1. The composition of the resulting corpus is reported in Section 5.2.

Preprocessing first standardizes carrier frequency, pulse width, and log-amplitude using statistics from the training split and converts the measured AOA into Euclidean direction components. Each pulse train is then divided into fixed-length windows. Absolute TOA is finally min–max mapped inside each window, so that the timing feature seen by the encoder depends only on pulses that co-occur in the same attention context. The encoder input is therefore the $d = 7$-dimensional vector of scaled carrier frequency, pulse width, log-amplitude, window-local TOA, and the three incidence-direction components.

No hand-crafted timing features are appended to this vector. Preliminary experiments augmented each pulse with lagged timing features, namely its instantaneous ΔTOA together with rolling means and standard deviations of ΔTOA over several look-back widths, to expose the local PRI structure explicitly; this configuration consistently degraded both emitter and mode clustering relative to the plain PDW input. A plausible explanation is that such look-back statistics summarize only the pulses preceding each position and mix contributions from all interleaved emitters, so under the pulse-count imbalance of the corpus (Section 5.2) they are dominated by the densest emitters and carry little signal for rare ones, whereas unmasked self-attention (Section 4.3) can relate each pulse to every other pulse in the window and recover inter-arrival structure per emitter. Letting the encoder learn timing structure implicitly also avoids committing to a fixed set of summary statistics, which keeps the representation free to adapt to interleaving patterns and PRI schedules that differ from those seen during training.

4.2.1 Per-feature scaling

Standardization statistics are estimated across all pulse trains in the training split and applied unchanged to validation and test data. Carrier frequency, pulse width, and log-amplitude are standardized per feature as

$$\tilde{g}_n = \frac{g_n - \mu_g}{\sigma_g}. \tag{4.6}$$

Here μ_g and σ_g are the corresponding training-set mean and standard deviation, and amplitude is log-transformed to reduce its dynamic range. The measured AOA consists of an azimuth θ_n^{az} and a latitude θ_n^{lat}, which together specify the incidence direction of the pulse on the unit sphere. Rather than feeding these angles to the encoder directly, each pair is converted to the Euclidean unit direction vector

$$\boldsymbol{u}_n = \left(u_n^x,\, u_n^y,\, u_n^z\right)^\top = \left(\cos\theta_n^{\mathrm{lat}}\cos\theta_n^{\mathrm{az}},\ \cos\theta_n^{\mathrm{lat}}\sin\theta_n^{\mathrm{az}},\ \sin\theta_n^{\mathrm{lat}}\right)^\top, \tag{4.7}$$

whose components lie in $[-1, 1]$ and are mapped to $[0, 1]$ by the fixed affine rescaling

$$\tilde{u}_n^c = \frac{u_n^c + 1}{2}, \qquad c \in \{x, y, z\}. \tag{4.8}$$

Neither map depends on the training corpus.

The Cartesian representation is chosen because angular coordinates are periodic and a scalar feature channel cannot express this: azimuths just below 2π and just above 0 describe nearly the same incidence direction, yet a direct rescaling of the angle places them at opposite ends of the feature range, creating a spurious discontinuity. The direction components of Equation (4.7) are free of this wrap-around, since nearby directions on the sphere always receive nearby coordinates. The same argument applies with greater force to the relative attention encodings of Section 4.4, which act on coordinate *differences*: a difference of rescaled azimuths does not respect the identification of 0 with 2π, whereas differences of direction components remain faithful to the underlying geometry.

Corpus-wide standardization of frequency, pulse width, and amplitude preserves absolute ranges that may distinguish operating modes across pulse trains. Absolute TOA is left in physical units at this stage and is min–max mapped only after windowing (Equation (4.10)).

4.2.2 Windowing

Standard self-attention has time and memory complexity $\mathcal{O}(L^2)$ for a window of L pulses. Because a complete pulse train may contain millions of pulses, the encoder processes fixed-length windows rather than entire trains. The value of L is reported with the experimental hyperparameters in Table 5.2.

Given a pulse train $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)$ and stride s, window w is

$$\boldsymbol{X}^{(w)} = \left(\boldsymbol{x}_{1+(w-1)s}, \ldots, \boldsymbol{x}_{L+(w-1)s}\right), \qquad w \in \left\{1, \ldots, \left\lfloor \frac{N-L}{s} \right\rfloor + 1\right\}. \tag{4.9}$$

The number of windows therefore varies with the pulse-train length. Any trailing segment shorter than L is dropped.

During training, choosing $s < L$ produces overlapping windows and therefore more training samples. A smaller stride improves coverage of sparse emitters and modes

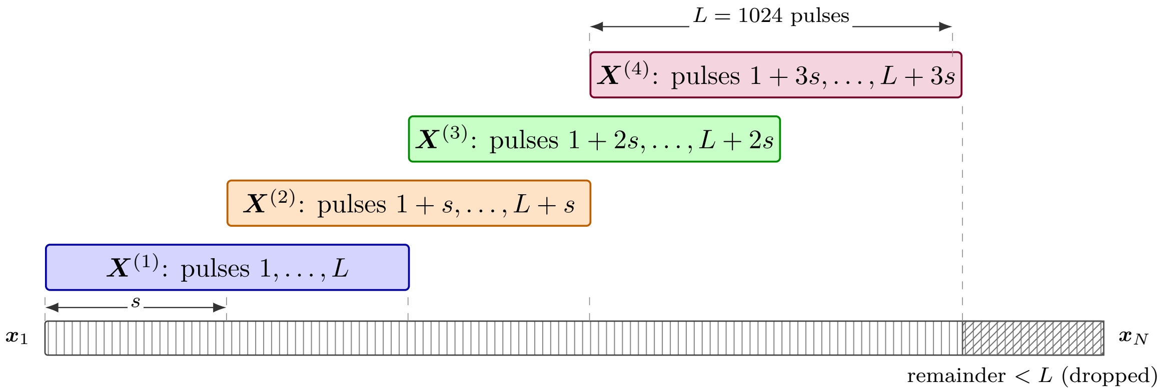

Figure 4.2. Sliding-window segmentation of a pulse train. Successive windows of length L advance by stride s and overlap by $L - s$ pulses. A trailing segment shorter than L is dropped.

and reduces sensitivity to window boundaries, but it also repeats more of the same PDWs, increasing computation and correlation between samples. The scenario-aware sampler in Section 4.6 allows the contrastive loss to use these repeated views. Advancing the window by a small stride resembles streaming inference because outputs can be refreshed as new pulses arrive, although it is not online learning because the model parameters remain fixed.

Validation and test use $s = L$, yielding non-overlapping windows that are processed entirely offline. Streaming inference and online model adaptation are left to future work (Section 7.3).

Each window retains the emitter label, mode label, and pulse-train identifier associated with every pulse. The emitter and mode labels supervise their respective branches and provide evaluation targets. The pulse-train identifier is not an encoder input or prediction target; it is used to keep train-local emitter identifiers distinct and to form scenario-aware mini-batches.

After windowing, absolute TOA is min–max mapped to $[0, 1]$ using only the L pulses inside the current window:

$$\tilde{t}_i^{(w)} = \frac{t_i^{(w)} - t_{\min}^{(w)}}{t_{\max}^{(w)} - t_{\min}^{(w)}}, \tag{4.10}$$

where $t_{\min}^{(w)}$ and $t_{\max}^{(w)}$ are the minimum and maximum physical TOA among the pulses of window w, and i indexes positions within that window. The encoder never observes pulses outside the window it processes, so recording-level extrema would inject timing information unavailable inside the attention context (and unavailable to a streaming receiver that buffers only the current window). Per-window mapping keeps the TOA channel self-contained; when training uses $s < L$, the same physical pulse may receive different normalized values in overlapping windows, which is the intended behaviour for a window-local feature.

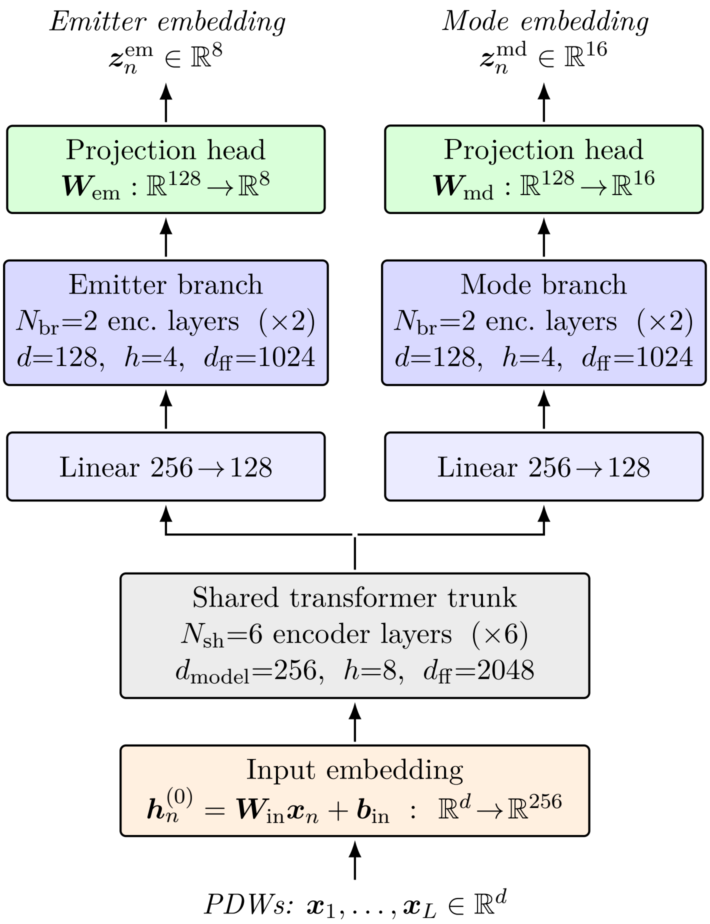

Figure 4.3. Architecture of the encoder $f_{\boldsymbol{\theta}}$. A sequence of PDWs is first lifted to $d_{\mathrm{model}} = 256$ by an input embedding, processed by a shared trunk of 6 transformer-encoder layers, and then split into two parallel branches with their own $256 \to 128$ projection and 2 encoder layers. Branch outputs are mapped by linear projection heads to the emitter embedding space $\mathbb{R}^8$ and the mode embedding space $\mathbb{R}^{16}$.

4.3 Transformer Encoder Architecture

The encoder $f_{\boldsymbol{\theta}}$ takes the form of a transformer-encoder backbone [11] with a shared trunk followed by two task-specific branches for mode clustering and deinterleaving. The overall layout is shown in Figure 4.3. The trunk produces a single contextualised representation of the input window, which the branches specialise into emitter-oriented and mode-oriented embeddings for the two clustering tasks defined in Section 4.1.

Input embedding

Each pulse vector $\boldsymbol{x}_n \in \mathbb{R}^d$, built from the scaled PDWs of Section 4.2, is mapped to a token embedding through an affine projection

$$\boldsymbol{h}_n^{(0)} = \boldsymbol{W}_{\mathrm{in}} \boldsymbol{x}_n + \boldsymbol{b}_{\mathrm{in}}, \qquad \boldsymbol{h}_n^{(0)} \in \mathbb{R}^{d_{\mathrm{model}}}, \tag{4.11}$$

where $\boldsymbol{W}_{\text{in}} \in \mathbb{R}^{d_{\text{model}} \times d}$ and d_{model} is the trunk width specified below. No positional encoding is added to $\boldsymbol{h}_n^{(0)}$. The temporal ordering of pulses is already carried by the TOA entry of $\boldsymbol{x}_n$, which makes a separate position signal redundant; consistent with this argument, Gunn et al. [5] report that adding sinusoidal or learnable positional encodings to a transformer deinterleaver yields a small but reproducible drop in performance, attributed to the resulting redundancy with the TOA feature.

Shared trunk

The token sequence $(\boldsymbol{h}_1^{(0)}, \ldots, \boldsymbol{h}_L^{(0)})$ is processed by a stack of N_{sh} standard transformer-encoder layers, each comprising a MHA sub-layer and a position-wise FFN sub-layer with residual connections and layer normalisation around both. Self-attention is unmasked, so every pulse can attend to every other pulse in the window. The trunk operates with hidden dimension $d_{\text{model}} = 256$, $h_{\text{sh}} = 8$ attention heads (so that each head has dimension 32), and an inner FFN dimension of $d_{\text{ff,sh}} = 2048$; $N_{\text{sh}} = 6$ layers are used. Its output is a contextualised token sequence $\boldsymbol{H}^{\text{sh}} = (\boldsymbol{h}_1^{\text{sh}}, \ldots, \boldsymbol{h}_L^{\text{sh}})$ with $\boldsymbol{h}_n^{\text{sh}} \in \mathbb{R}^{256}$, in which information has been mixed across the full window.

Task-specific branches

The trunk output feeds two parallel branches that do not share parameters. Each branch begins with an affine projection that reduces the hidden width from $d_{\text{model}} = 256$ to $d_{\text{br}} = 128$, after which $N_{\text{br}} = 2$ transformer-encoder layers of the same form as the trunk are applied. Within a branch, MHA uses $h_{\text{br}} = 4$ heads (again with per-head dimension 32) and the FFN inner dimension is $d_{\text{ff,br}} = 1024$. The reduced width and depth reflect a deliberate asymmetry: the trunk is intended to carry the heavy lifting of contextual mixing, while the branches only need to specialise the representation to either emitter identity or operating mode. Write the resulting token sequences as $\boldsymbol{H}^{\text{em}} = (\boldsymbol{h}_1^{\text{em}}, \ldots, \boldsymbol{h}_L^{\text{em}})$ and $\boldsymbol{H}^{\text{md}} = (\boldsymbol{h}_1^{\text{md}}, \ldots, \boldsymbol{h}_L^{\text{md}})$ with $\boldsymbol{h}_n^{\text{em}}, \boldsymbol{h}_n^{\text{md}} \in \mathbb{R}^{128}$.

Projection heads

Each branch terminates in a linear projection head that maps every token to its respective embedding space,

$$\boldsymbol{z}_n^{\text{em}} = \boldsymbol{W}_{\text{em}} \boldsymbol{h}_n^{\text{em}} + \boldsymbol{b}_{\text{em}}, \qquad \boldsymbol{z}_n^{\text{md}} = \boldsymbol{W}_{\text{md}} \boldsymbol{h}_n^{\text{md}} + \boldsymbol{b}_{\text{md}}, \tag{4.12}$$

with $\boldsymbol{z}_n^{\text{em}} \in \mathbb{R}^p$ and $\boldsymbol{z}_n^{\text{md}} \in \mathbb{R}^q$ for $p = 8$ and $q = 16$. Collecting the per-pulse outputs gives the two embedding sequences

$$\boldsymbol{Z}^{\text{em}} = (\boldsymbol{z}_1^{\text{em}}, \ldots, \boldsymbol{z}_L^{\text{em}}), \qquad \boldsymbol{Z}^{\text{md}} = (\boldsymbol{z}_1^{\text{md}}, \ldots, \boldsymbol{z}_L^{\text{md}}). \tag{4.13}$$

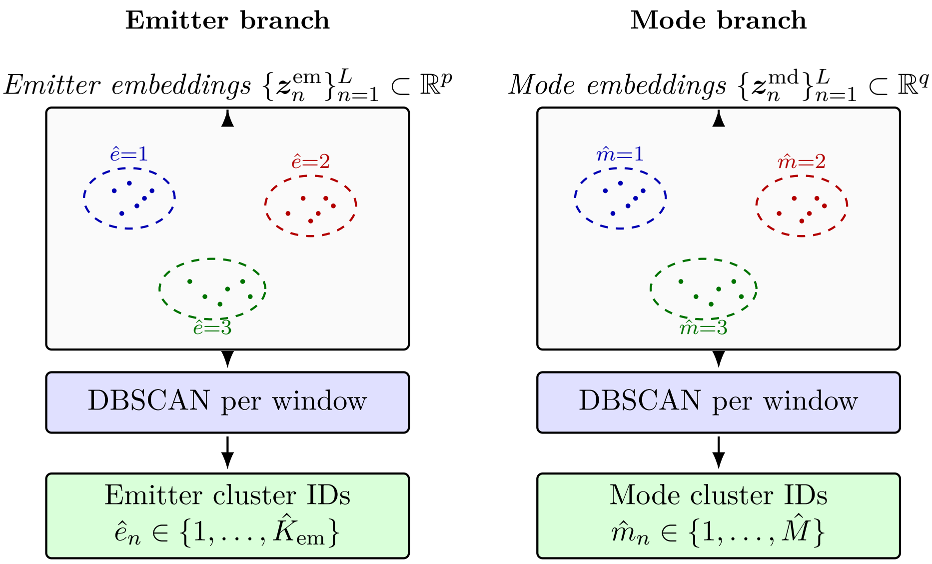

Figure 4.4. Inference-time decoding of branch embeddings into per-pulse cluster indices via Equation (4.14). Dots are pulse embeddings and dashed ellipses are DBSCAN clusters in one window. The pipeline is identical on both branches and the integer outputs are local to each clustering instance; they acquire their interpretation from the supervision scope of Section 4.1, which differs between the two branches.

The mode space is assigned the higher dimensionality because, conditional on emitter identity, mode structure is expected to require a richer geometry to separate subtle waveform regimes, whereas emitter membership induces a coarser partition over the same pulses and is well served by a more compact embedding.

From continuous embeddings to discrete partitions

The projection heads in Equation (4.12) produce continuous vectors, whereas the per-pulse outputs in Equations (4.3) and (4.4) are discrete. At inference time, each branch's L-token embedding sequence is therefore decoded independently and per window by a fixed clustering operator,

$$(\hat{e}_1, \ldots, \hat{e}_L) = \mathrm{DBSCAN}\Big(\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^{L}\Big), \quad (\hat{m}_1, \ldots, \hat{m}_L) = \mathrm{DBSCAN}\Big(\{\boldsymbol{z}_n^{\mathrm{md}}\}_{n=1}^{L}\Big), \quad (4.14)$$

with cardinalities $\hat{K}, \hat{M} \in \mathbb{N}$ inferred from the data (Figure 4.4). DBSCAN [4] groups embeddings by local density at a fixed Euclidean radius $\varepsilon = 0.4$ and minimum neighbourhood count $\mathrm{minPts} = 5$, assigning sparser points to a noise category. Clustering per window matches the train-local supervision scope of the emitter branch and keeps the runtime linear in window length; cross-window alignment of mode embeddings is instead supplied by the batch-wide contrastive term of Section 4.5 and resolved by the permutation-invariant metrics of Section 4.7. The operator contributes no gradients, and the trunk, branch, and projection-head weights together form the parameter vector $\boldsymbol{\theta}$ optimised by the loss in Section 4.5.

4.4 Attention Variants

The encoder of Section 4.3 uses standard self-attention, which treats a window as an unordered set of tokens and lets every pulse attend to every other pulse purely through learned query–key compatibility. The only ordering or physical-relationship cue available to it is whatever the input features themselves carry, in particular the TOA entry of each PDW. A radar pulse train, however, has strong pairwise structure: two pulses that are close in time, share a carrier frequency, or have a similar pulse width are far more likely to originate from the same emitter or mode than an arbitrary pair. This section describes two variants of the attention mechanism that make such structure explicit. Each is evaluated as a drop-in replacement for the MHA sub-layers of Section 4.3; their relative merit is reported in Chapter 5.

Throughout, we write the attention logits of a single head over a window of L pulses as

$$\boldsymbol{A} = \frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}, \qquad A_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}}, \tag{4.15}$$

where $\boldsymbol{q}_i$ and $\boldsymbol{k}_j$ are the query and key of pulses i and j and d_k is the per-head dimension. The attention weights are the row-wise softmax of $\boldsymbol{A}$, which then mixes the value vectors.

Physical attention bias

The first variant adds a handcrafted pairwise bias to the logits, computed from the physical parameters of each pulse pair. For each physical feature f of interest, a fixed term $B_{ij}^{(f)}$ is computed from the values of pulses i and j, and the terms are superposed with learnable scalar coefficients,

$$A_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}} + \sum_f \lambda_f \, B_{ij}^{(f)}, \tag{4.16}$$

where each coefficient λ_f is learned per head jointly with the rest of the network. The features of primary interest are the TOA and the arrival direction: $B_{ij}^{(f)}$ is a fixed function of the time separation $|t_i - t_j|$ or of the direction separation $\|\boldsymbol{u}_i - \boldsymbol{u}_j\|$ between the unit direction vectors of Equation (4.7), chosen so that pulses close in time or arriving from similar directions receive a larger value. The chordal distance $\|\boldsymbol{u}_i - \boldsymbol{u}_j\|$ is monotone in the angle enclosed between the two directions and is unaffected by azimuth wrap-around, unlike a difference of raw angles (Section 4.2.1). This pushes attention towards physically related pairs before any learning takes place, while the coefficients λ_f let the model weight each cue—or suppress it entirely—during training instead of committing to hand-tuned relative strengths. The construction mirrors the relative-position and content-independent biases used in language and graph transformers [15, 16], specialised here to pulse parameters rather than token indices.

Rotary positional encoding

The bias variant adjusts the logits after the query–key product is formed; the second variant instead builds positional information into the queries and keys themselves. Following Su et al. [17], let the query and key of a pulse be functions of both its hidden representation $\boldsymbol{h}_i$ entering the attention sub-layer and a scalar coordinate p_i attached to the pulse, and require that their inner product depend on the coordinates only through their difference,

$$\boldsymbol{q}_i = f_q(\boldsymbol{h}_i, p_i), \qquad \boldsymbol{k}_j = f_k(\boldsymbol{h}_j, p_j), \qquad \boldsymbol{q}_i^\top \boldsymbol{k}_j = g(\boldsymbol{h}_i, \boldsymbol{h}_j, p_j - p_i). \tag{4.17}$$

In the original formulation the coordinate is the token index, $p_i = i$. We instead attach raw pulse parameters, so that the difference $p_j - p_i$ is a physical quantity—the time gap or the difference in a direction component between two pulses—rather than an ordinal distance; this suits the irregularly sampled pulse trains of Section 4.2, where consecutive indices may be microseconds or milliseconds apart.

RoPE satisfies Equation (4.17) by encoding the coordinate as a phase. The d_k dimensions of a head (assumed even) are grouped into $d_k/2$ two-dimensional planes, and the m-th plane is rotated by the angle $p\theta_m$ using the planar rotation matrix

$$\boldsymbol{\Omega}(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}. \tag{4.18}$$

Collecting the per-plane rotations into a single block-diagonal matrix gives

$$\boldsymbol{R}(p) = \operatorname{diag}\Big(\boldsymbol{\Omega}(p\theta_1),\, \boldsymbol{\Omega}(p\theta_2),\, \dots,\, \boldsymbol{\Omega}(p\theta_{d_k/2})\Big), \qquad \theta_m = b^{-2(m-1)/d_k}, \tag{4.19}$$

with a fixed base b (we use $b = 10^4$, following Su et al. [17]). The geometric progression of angular frequencies θ_m mirrors the wavelengths of the sinusoidal encoding of Vaswani et al. [11]: fast-rotating planes resolve fine coordinate differences, slow-rotating planes capture coarse ones. The query and key functions are then the usual linear projections followed by this rotation,

$$f_q(\boldsymbol{h}_i, p_i) = \boldsymbol{R}(p_i)\, \boldsymbol{W}_q \boldsymbol{h}_i, \qquad f_k(\boldsymbol{h}_j, p_j) = \boldsymbol{R}(p_j)\, \boldsymbol{W}_k \boldsymbol{h}_j, \tag{4.20}$$

written $\tilde{\boldsymbol{q}}_i$ and $\tilde{\boldsymbol{k}}_j$ below.

That this construction meets the design goal follows from one property of planar rotations: composing two of them subtracts their angles, $\boldsymbol{\Omega}(\phi)^\top \boldsymbol{\Omega}(\psi) = \boldsymbol{\Omega}(\psi - \phi)$, hence block-wise $\boldsymbol{R}(p_i)^\top \boldsymbol{R}(p_j) = \boldsymbol{R}(p_j - p_i)$ and

$$\tilde{\boldsymbol{q}}_i^\top \tilde{\boldsymbol{k}}_j = (\boldsymbol{W}_q \boldsymbol{h}_i)^\top\, \boldsymbol{R}(p_j - p_i)\, (\boldsymbol{W}_k \boldsymbol{h}_j), \tag{4.21}$$

which is exactly the form required by Equation (4.17): each pulse is rotated by its absolute coordinate, yet the logit sees only the difference. Figure 4.6 illustrates this on a single plane: increasing the coordinate fans a vector around the circle at a fixed rate (left), while the phase enclosed between a rotated query and key—and hence their dot product—is set by the coordinate gap alone (right).

The coordinates we investigate are the TOA t_n and the three incidence-direction components u_n^x, u_n^y, and u_n^z of Equation (4.7), normalised as in Section 4.2. The shared trunk of Section 4.3 and the mode branch use a single coordinate: every head rotates all of its $d_k/2$ planes by the TOA, applying $\boldsymbol{R}(t_n)$ of Equation (4.19) directly. The arrival direction is deliberately excluded there, because an operating mode is characterised by the intrinsic temporal pattern of the emission and must be recognised regardless of the direction from which it is observed; rotating the mode branch by the arrival direction would inject nuisance variation into an embedding that should be invariant to geometry.

For the deinterleaving branch, by contrast, direction is a strong cue: within a window, pulses from one emitter arrive from a nearly constant direction. This branch therefore encodes all four coordinates at once by partitioning the $d_k/2$ planes of each head into four disjoint slices and rotating each slice by its own coordinate, in the manner of the multi-axis RoPE used for vision transformers [18],

$$\boldsymbol{R}_n^{\mathrm{em}} = \operatorname{diag}\Big(\boldsymbol{R}^{(1)}(t_n),\ \boldsymbol{R}^{(2)}(u_n^x),\ \boldsymbol{R}^{(3)}(u_n^y),\ \boldsymbol{R}^{(4)}(u_n^z)\Big), \tag{4.22}$$

where slice g comprises c_g planes with $c_1 + c_2 + c_3 + c_4 = d_k/2$, and $\boldsymbol{R}^{(g)}$ stacks c_g planar rotations with its own frequency ladder exactly as in Equation (4.19). Because the slices occupy disjoint dimensions of the query and key, the logit of Equation (4.21) splits into a sum of per-coordinate relative terms,

$$\begin{aligned} \tilde{\boldsymbol{q}}_i^\top \tilde{\boldsymbol{k}}_j = \boldsymbol{q}_i^{(1)\top} \boldsymbol{R}^{(1)}(t_j - t_i)\, \boldsymbol{k}_j^{(1)} + \boldsymbol{q}_i^{(2)\top} \boldsymbol{R}^{(2)}\Big(u_j^x - u_i^x\Big)\, \boldsymbol{k}_j^{(2)} \\ + \boldsymbol{q}_i^{(3)\top} \boldsymbol{R}^{(3)}\Big(u_j^y - u_i^y\Big)\, \boldsymbol{k}_j^{(3)} + \boldsymbol{q}_i^{(4)\top} \boldsymbol{R}^{(4)}\Big(u_j^z - u_i^z\Big)\, \boldsymbol{k}_j^{(4)}, \end{aligned} \tag{4.23}$$

with $\boldsymbol{q}_i^{(g)}$ and $\boldsymbol{k}_j^{(g)}$ the sub-vectors of the unrotated query and key in slice g. The first term compares pulses by their time gap and the remaining three by the difference in each direction component, so time and direction contribute independent, additively superposed relative phases to the same attention logit. Figure 4.5 contrasts the two layouts. The Cartesian direction components of Equation (4.7) matter particularly in this role: because RoPE turns coordinate *differences* into relative phases, rotating by a rescaled azimuth would ignore the identification of 0 with 2π, so two pulses arriving from nearly the same direction could receive an almost maximal relative phase; differences of direction components instead vary continuously over the sphere.

This variant complements rather than replaces the bias term—the bias injects pairwise relationships additively into the logits, while RoPE embeds coordinate differences in the geometry of the queries and keys—and it revisits the choice of Section 4.3 to rely on TOA and the arrival direction as plain input features, testing whether an explicit relative encoding adds information beyond their presence in the input.

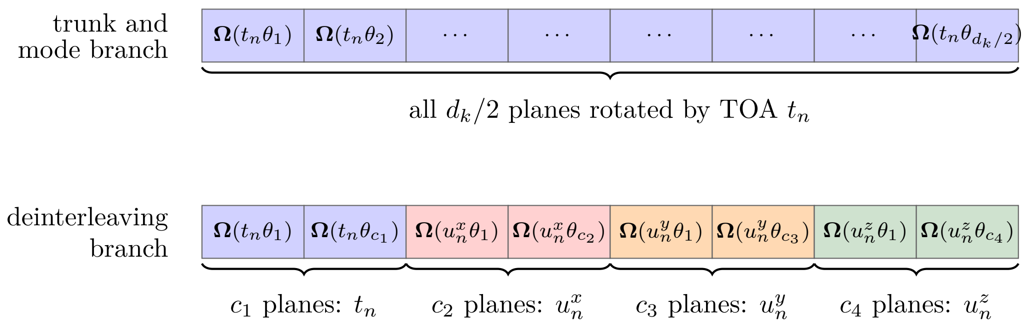

Figure 4.5. Allocation of the $d_k/2$ rotation planes of one attention head to physical coordinates. In the shared trunk and the mode branch (top), every plane is rotated by the TOA t_n down its frequency ladder (Equation (4.19)); the arrival direction is excluded because the mode embedding should be invariant to viewing geometry. In the deinterleaving branch (bottom), the planes are partitioned into four disjoint slices of sizes c_1 through c_4, rotated by t_n, u_n^x, u_n^y, and u_n^z respectively (Equation (4.22)), so that the attention logit decomposes into the per-coordinate relative terms of Equation (4.23).

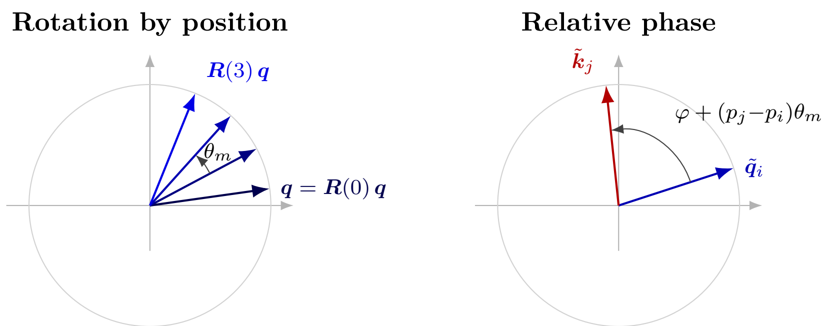

Figure 4.6. Geometry of RoPE on one of the $d_k/2$ two-dimensional planes of a head. *Left:* increasing the coordinate p rotates a query vector $\boldsymbol{q}$ at a fixed rate θ_m (Equation (4.19)), so the absolute coordinate is encoded as an absolute phase; here p takes the integer values $0, \ldots, 3$, but any real coordinate such as the TOA is admissible. *Right:* the angle enclosed between a rotated query $\tilde{\boldsymbol{q}}_i$ and a rotated key $\tilde{\boldsymbol{k}}_j$ equals the content angle plus $(p_j - p_i)\theta_m$; since the inner product depends only on this enclosed angle, the attention logit is a function of the coordinate difference $p_j - p_i$ (Equation (4.21)).

4.5 Loss Function

The training objective is a sum of two *supervised contrastive* [19] terms with the same functional form, both aggregated over the whole mini-batch. The two terms differ only in which label drives the positive set: an emitter identifier on the emitter branch, and the global mode label of Section 4.1 on the mode branch. Routing the mode branch through a contrastive loss rather than a closed-catalogue softmax keeps the inference pipeline of Equation (4.14) open to modulations absent from training; the post-hoc clustering operator contributes no gradients and is not part of the objective. A further consequence of this design is that the pre-projection encodings remain general-purpose features: supervised classification heads (for example, against a library of known emitters) can later be attached to the same base model without retraining the encoder.

Supervised contrastive kernel

All projection-head outputs of Equation (4.12) are ℓ_2-normalised, so that the cosine similarity reduces to the inner product

$$s_{ij} \;\coloneqq\; \boldsymbol{z}_i^\top \boldsymbol{z}_j \;\in\; [-1, 1], \tag{4.24}$$

and a fixed temperature $\tau > 0$ rescales it inside the loss; Smaller τ concentrates the gradient on the hardest negatives, while larger τ stabilizes early optimization, since $\|\nabla \mathcal{L}_{\mathrm{SC}}\| \propto \frac{1}{\tau}$ [19]. Both branches use $\tau = 0.2$, the value commonly chosen for normalised contrastive features [9, 19].

For an anchor set I with embeddings $\{\boldsymbol{z}_i\}_{i \in I}$, write the candidate set $A(i) = I \setminus \{i\}$ and the positive subset $P(i) \subseteq A(i)$ defined by label coincidence. The supervised contrastive contribution [19] is

$$\mathcal{L}_{\mathrm{SC}}\Big(I, \{P(i)\}\Big) \;=\; -\sum_{i \in I^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(s_{ip}/\tau)}{\sum_{a \in A(i)} \exp(s_{ia}/\tau)}, \tag{4.25}$$

where $I^+ = \{i \in I : P(i) \neq \emptyset\}$ and $\mathcal{L}_{\mathrm{SC}} = 0$ when $I^+ = \emptyset$. Anchors outside I^+ contribute only as negatives. The denominator runs over every candidate in $A(i)$, so unrelated samples exert repulsive pressure and the encoder cannot collapse to a constant map [9].

Branch losses and total objective

Both branches apply Equation (4.25) to the same anchor set, namely all $B \times L$ pulses of a mini-batch pooled across windows, and differ only in the label that defines the positive sets: two pulses are positives on the emitter branch when they originate from the same physical emitter, and on the mode branch when they carry the same global mode label. Since emitter identifiers are train-local in Section 4.1, they are made unique within a mini-batch by pairing each identifier with its originating pulse-train. Writing $\mathcal{L}_{\text{em}}$ and $\mathcal{L}_{\text{md}}$ for the resulting losses on the emitter and mode projections, the total training objective is

$$\mathcal{L} \;=\; \mathcal{L}_{\text{em}} + \lambda_{\text{md}}\,\mathcal{L}_{\text{md}}, \qquad \lambda_{\text{md}} > 0, \tag{4.26}$$

with λ_{md} reported in Chapter 5.

Probabilistic interpretation

The kernel Equation (4.25) admits a Gibbs–Boltzmann reading that clarifies the role of every factor. For each anchor $i \in I^+$, define the categorical distribution over its candidates

$$P_{\boldsymbol{\theta}}(a \mid i) \;=\; \frac{\exp(s_{ia}/\tau)}{Z_i}, \qquad Z_i \;=\; \sum_{a' \in A(i)} \exp(s_{ia'}/\tau), \qquad a \in A(i), \tag{4.27}$$

which is exactly the Boltzmann distribution with energy $-s_{ia}$ and temperature τ on the configuration space $A(i)$. Substituting Equation (4.27) into Equation (4.25) yields the per-anchor cross-entropy between the empirical positive distribution (uniform on $P(i)$) and the model $P_{\boldsymbol{\theta}}(\cdot \mid i)$,

$$\mathcal{L}_{\text{SC}} \;=\; -\sum_{i \in I^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \log P_{\boldsymbol{\theta}}(p \mid i), \tag{4.28}$$

that is, minus the average log-evidence the encoder assigns to the positives, exactly as a maximum-likelihood estimator would treat $P(i)$ as a sample of observations drawn from the latent positive class.

Applying $\log(x/y) = \log x - \log y$ to Equation (4.25) separates the attractive and repulsive contributions explicitly:

$$\mathcal{L}_{\text{SC}} \;=\; \underbrace{\sum_{i \in I^+} \log Z_i}_{\text{log-partition (repulsion)}} \;-\; \underbrace{\sum_{i \in I^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \frac{s_{ip}}{\tau}}_{\text{mean positive similarity (attraction)}}. \tag{4.29}$$

The first term is monotone in similarities to every candidate, with $\partial \log Z_i / \partial s_{ia} = P_{\boldsymbol{\theta}}(a \mid i)/\tau$, so the gradient automatically concentrates on the hardest negatives—those whose Boltzmann weight under Equation (4.27) is largest despite carrying a different label. The second term rewards alignment of each anchor with its positives, with each positive contributing $1/(\tau|P(i)|)$ to the gradient.

The prefactor $1/|P(i)|$ turns the inner sum of log-probabilities into an arithmetic mean, equivalently into the log of a geometric mean,

$$-\frac{1}{|P(i)|} \sum_{p \in P(i)} \log P_{\boldsymbol{\theta}}(p \mid i) \;=\; -\log\left(\prod_{p \in P(i)} P_{\boldsymbol{\theta}}(p \mid i) \right)^{1/|P(i)|}. \tag{4.30}$$

Without the prefactor, the inner sum equals the log-likelihood of $|P(i)|$ conditionally independent observations of the latent positive class, and the loss would scale extensively with the number of available positives. Dividing by $|P(i)|$ replaces this product by its geometric mean, so an anchor's contribution becomes intensive in the size of its positive set; this is what allows the same kernel to remain well-behaved across the two branches, where $|P(i)|$ ranges from a handful (emitter contrasts inside one window) to many hundreds (mode contrasts pooled over the batch).

4.6 Training Procedure

Parameters $\boldsymbol{\theta}$ are estimated by minimising $\mathcal{L}$ (Equation (4.26)) with Adam [20]. Learning rate, weight decay, epoch count, dropout, and related settings are listed in Table 5.2.

Scenario-aware batch sampling

Mini-batches are not formed by uniform shuffling of the pooled windows. Overlapping windows from Section 4.2.2 must co-occur in a batch to act as shifted views under the contrastive objective, and emitter identifiers are train-local (Section 4.1), so cross-window emitter positives exist only within a scenario. Each mini-batch is therefore assembled from a small number of scenarios, each contributing several (possibly overlapping) windows. Within a scenario group the emitter branch obtains cross-window positives; across groups the mode branch obtains batch-wide negatives and cross-scenario mode positives as required by Section 4.5. The number of groups and windows per group, together with any λ_{md} warm-up or early-stopping policy, are recorded in Table 5.2.

4.7 Evaluation Metrics

The methods of Sections 4.3 and 4.5 return per-pulse cluster indices $\hat{y}_n$ that have to be compared with reference labels y_n. Both labellings partition the same index set $\{1, \ldots, N\}$, instantiated by either the emitter pair $(e_n, \hat{e}_n)$ within a window or the mode pair $(m_n, \hat{m}_n)$ across windows, as in Equations (4.1), (4.2) and (4.13). The integers used to name each cluster are arbitrary and may differ between the two labellings, so every metric reported here is invariant to relabelling: only the induced partitions matter, not the symbols themselves. This is essential because, by construction, the clustering operator of Equation (4.14) produces window-local integers that bear no fixed relation to the ground-truth catalogue.

All metrics derive from the contingency table

$$n_{ij} = \sum_{n=1}^{N} \mathbf{1}[y_n = i,\ \hat{y}_n = j],$$

with marginals $a_i = \sum_j n_{ij}$ and $b_j = \sum_i n_{ij}$ counting how many pulses carry true label i

or predicted label j. No single scalar captures every facet of cluster agreement, so we report metrics drawn from three different families — pair counting, information theory, and entropy-based homogeneity/completeness — together with a Hungarian-aligned accuracy when the two cardinalities are comparable.

Pair-counting view: Rand and adjusted Rand

The most intuitive way to compare two partitions is to look at every *pair* of pulses and ask whether the two labellings agree on a binary question: are these two pulses in the *same* cluster, or in *different* clusters? Each unordered pair $\{m, n\}$ falls into one of four categories: same in both, same in Y but different in $\hat{Y}$, different in Y but same in $\hat{Y}$, or different in both. The first and last are agreements; the middle two are disagreements. A perfectly aligned pair of partitions has only agreements; a random pair has many disagreements.

It is convenient to count pairs that are placed *together* by each partition. Writing $S_{Y\hat{Y}}$, S_Y, and $S_{\hat{Y}}$ for the numbers of pairs that are together in both partitions, in Y, and in $\hat{Y}$ respectively, the contingency table gives

$$S_{Y\hat{Y}} = \sum_{i,j} \binom{n_{ij}}{2}, \qquad S_Y = \sum_{i} \binom{a_i}{2}, \qquad S_{\hat{Y}} = \sum_{j} \binom{b_j}{2}, \tag{4.31}$$

out of $\binom{N}{2}$ total pairs. The plain Rand index [21] is the fraction of pairs on which the two partitions agree,

$$R(Y, \hat{Y}) = 1 - \frac{S_Y + S_{\hat{Y}} - 2\, S_{Y\hat{Y}}}{\binom{N}{2}} \in [0, 1]. \tag{4.32}$$

The Rand index is easy to read but inflated: when most pulses sit in different clusters, the bulk of pairs are correctly judged “different in both” simply by chance, so R stays close to one even for poorly aligned partitions. The ARI [21] corrects this by subtracting the expected value of $S_{Y\hat{Y}}$ under a permutation null that fixes the cluster sizes $\{a_i\}$ and $\{b_j\}$ but otherwise reassigns pulses uniformly,

$$\mathbb{E}[S_{Y\hat{Y}}] = \frac{S_Y\, S_{\hat{Y}}}{\binom{N}{2}}. \tag{4.33}$$

Subtracting this expectation from numerator and denominator and rescaling so that perfect agreement gives one yields

$$\mathrm{ARI} = \frac{S_{Y\hat{Y}} - \mathbb{E}[S_{Y\hat{Y}}]}{\frac{1}{2}\Big(S_Y + S_{\hat{Y}}\Big) - \mathbb{E}[S_{Y\hat{Y}}]}, \qquad \mathrm{ARI} \le 1. \tag{4.34}$$

The numerator is the excess of together-together pairs over chance; the denominator is the largest such excess achievable while preserving the marginals. Hence $\mathrm{ARI} = 1$

at exact recovery up to relabelling, ARI ≈ 0 at chance, and negative values indicate worse-than-random alignment.

Information-theoretic view: adjusted mutual information

A second viewpoint asks how much knowing one labelling tells us about the other: if $\hat{Y}$ is a deterministic function of Y, then announcing $\hat{Y}$ leaves no remaining uncertainty about Y, and conversely two independent labellings are uninformative about each other. This is exactly what mutual information measures [22]. With Shannon entropies of the marginals

$$H(Y) = -\sum_{i:\, a_i>0} \frac{a_i}{N} \log \frac{a_i}{N}, \qquad H(\hat{Y}) = -\sum_{j:\, b_j>0} \frac{b_j}{N} \log \frac{b_j}{N}, \tag{4.35}$$

quantifying the marginal uncertainty of each labelling, the empirical mutual information is

$$\mathrm{MI}(Y;\hat{Y}) = \sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{N\, n_{ij}}{a_i b_j}, \tag{4.36}$$

with the convention $0 \log 0 = 0$. MI is non-negative, zero when Y and $\hat{Y}$ are independent, and equal to either marginal entropy when one labelling determines the other.

Like the Rand index, MI has a non-zero baseline expectation, and that expectation grows with the number of clusters. The AMI [22] performs the analogous chance correction: it subtracts the expected mutual information under a permutation null that fixes the cluster sizes $\{a_i\}$ and $\{b_j\}$, then rescales by the average of the marginal entropies,

$$\mathrm{AMI}(Y;\hat{Y}) = \frac{\mathrm{MI}(Y;\hat{Y}) - \mathbb{E}[\mathrm{MI}]}{\frac{1}{2}\Big(H(Y) + H(\hat{Y})\Big) - \mathbb{E}[\mathrm{MI}]}, \tag{4.37}$$

when the denominator is positive; otherwise AMI $\equiv 0$. Hence AMI $= 1$ at exact recovery up to relabelling, AMI ≈ 0 at chance, and negative values indicate worse-than-random alignment. AMI inherits the same range and chance baseline as ARI but operates on the information-theoretic axis, which makes it less sensitive to differences in cluster size and more sensitive to fine-grained mislabellings.

Homogeneity, completeness, and V-measure

A third viewpoint asks two complementary questions about cluster purity: *homogeneity*, "does each predicted cluster contain only one true class?", and *completeness*, "is each true class collected into a single predicted cluster?". Splitting a true class across many predicted clusters preserves homogeneity but destroys completeness; merging several true classes into one predicted cluster preserves completeness but destroys homogeneity. Reporting both quantities separately therefore diagnoses whether the encoder oversplits or undermerges.

With the conditional entropies

$$H(Y \mid \hat{Y}) = -\sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{b_j}, \qquad H(\hat{Y} \mid Y) = -\sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{a_i}, \tag{4.38}$$

homogeneity and completeness are [23]

$$h = 1 - \frac{H(Y \mid \hat{Y})}{H(Y)}, \qquad c = 1 - \frac{H(\hat{Y} \mid Y)}{H(\hat{Y})}, \tag{4.39}$$

with $h = 1$ if $H(Y) = 0$ and $c = 1$ if $H(\hat{Y}) = 0$. The V-measure is the harmonic mean

$$\mathrm{V}(Y, \hat{Y}) = \frac{2hc}{h+c}, \qquad h + c > 0, \tag{4.40}$$

and $\mathrm{V} = 0$ if $h = c = 0$. The harmonic mean is small whenever either component is small, so a high V-measure requires both homogeneity and completeness to be high simultaneously.

Clustering accuracy with Hungarian alignment

The most directly interpretable scalar is the fraction of pulses correctly classified, but the cluster integers are arbitrary so a one-to-one relabelling of $\hat{Y}$ onto Y must be chosen first. Among all such relabellings, the one that maximises the number of correct assignments is the optimal solution of a linear assignment problem on the contingency table $\{n_{ij}\}$, which the Hungarian algorithm [24] solves in polynomial time. The ACC is the resulting fraction,

$$\mathrm{ACC}(Y, \hat{Y}) = \frac{1}{N} \max_{\pi} \sum_{n=1}^{N} \mathbf{1}[\, y_n = \pi(\hat{y}_n)\,], \tag{4.41}$$

where the maximum is taken over injective maps π between predicted and true cluster identifiers; when the two cardinalities differ, the contingency table is zero-padded so that π remains injective. $\mathrm{ACC} \in [0, 1]$, with $\mathrm{ACC} = 1$ at perfect recovery up to relabelling. Compared with the other metrics, ACC is the most familiar quantity but the least forgiving when $\hat{K}$ deviates strongly from the true number of clusters, because unmatched predicted clusters then contribute only zeros to the maximum.

What each metric does and does not tell us

The four metrics agree at the extremes — all return one at exact recovery and approximately zero at chance — but they disagree in the middle. ARI summarises the pair structure and is comparable across runs with different cluster counts. AMI captures how much information one labelling carries about the other and remains comparable when the number of clusters varies. Reporting homogeneity and completeness alongside the

V-measure separates oversplitting from undermerging, which matters for the open-set mode setting where $\hat{M} > M$ is expected. ACC is reported when $\hat{K}$ and the true number of clusters are comparable; it is the easiest number to communicate but the most fragile when cardinalities diverge. We therefore report several scores rather than a single headline metric.

Qualitative inspection

Two-dimensional projections of $\{\boldsymbol{z}_n^{\mathrm{em}}\}$ and $\{\boldsymbol{z}_n^{\mathrm{md}}\}$ coloured by (e_n, m_n) complement the quantitative scores of Equations (4.34), (4.37), (4.40) and (4.41); protocols and numbers appear in Chapter 5.

4.8 Baseline Methods

The baselines isolate the contribution of the dual-branch layout in Section 4.3 and of the mode term $\lambda_{\mathrm{md}}\mathcal{L}_{\mathrm{md}}$ in Equation (4.26) relative to a single-stack encoder and to classical clustering without a sequence model. Final configurations and the full list of comparisons are deferred to Chapter 5; this section fixes only the design intent of each.

Single-stack transformer. A flat encoder of N_{ss}=8 transformer-encoder layers with the same width, heads, and FFN expansion as the trunk in Section 4.3, terminated by one linear head $\mathbb{R}^{d_{\mathrm{model}}} \to \mathbb{R}^p$ and trained on $\mathcal{L}_{\mathrm{em}}$ alone. Tests whether a single representation optimised purely for window-level deinterleaving also serves the mode clustering.

DBSCAN on fixed window features. No transformer; DBSCAN [4] clusters a deterministic vector built from the scaled PDWs of a window (vectorisation specified in Chapter 5). The neighbourhood radius ε is selected by grid search on the validation set, holding minPts fixed; the selected ε is then frozen for the held-out test evaluation. Separates the gain from learned sequence context from what density clustering already recovers on hand-specified summaries.

CHAPTER 5

Experiments and Results

5.1 Experimental Setup

All neural models in this chapter were trained and evaluated on a single workstation with one NVIDIA A100 GPU (80 GB high-bandwidth memory; some system utilities report a capacity closer to 82 GB because of binary versus decimal gigabyte conventions). The implementation uses Python with PyTorch; exact package versions are pinned in the environment file shipped with the thesis code repository when that repository is published.

Random seeds control weight initialization, training-window shuffling each epoch, and stochastic augmentations. Unless an experiment explicitly varies the seed, the same seed set is reused so that ablations in Section 5.5 are comparable under identical noise conditions. Concrete seed integers, per-run overrides, and mean $\pm$ standard deviation over multiple seeds for quantitative scores are reported with the tables in Section 5.4.

5.2 Datasets

All experiments in this chapter use synthetic PDW streams generated by the proprietary scenario simulator introduced in Section 4.2. The simulator supplies time-ordered pulse records together with ground-truth labels for emitter identity (meaningful only within one scenario, in the sense of Section 4.1) and operating mode (drawn from the global catalogue); both label streams are consumed by the training objective in Section 4.5 and are also used to compute the evaluation metrics in Sections 4.7 and 5.4.

Each *scenario* is one simulated timeline: a sequence of PDWs whose length and the number of concurrent emitters vary across draws so that the encoder sees both sparse and crowded electromagnetic pictures. Scenarios are disjointly assigned to training, validation, and test partitions before any windowing; training-split statistics for carrier frequency, pulse width, and amplitude in Section 4.2 are fit on the training partition only and applied unchanged to validation and test pulses, whereas TOA is min–max mapped independently inside each window. Windows contain $L = 2000$ pulses; training windows use stride $s = 200$, so consecutive windows overlap by $L - s = 1800$ pulses, whereas validation and test windows use stride $s = L$ and are therefore non-overlapping. As a consequence, the effective number of windows per scenario differs by split even when raw pulse counts are similar.

Table 5.1 summarises corpus-level figures. The training split contains 1461 scenarios and roughly 13.8 million pulses in total, while the validation and test splits contribute

Table 5.1. Synthetic scenario corpus used for training and evaluation. Per-scenario quantities are reported as mean ± standard deviation over the scenarios in each split.

		Per scenario (mean ± std)		
Split	Scenarios	Pulses	Emitters	Modes
Train	1461	9480 ± 1690	8.80 ± 4.33	4.33 ± 1.40
Validation	245	9180 ± 2240	8.59 ± 4.41	4.28 ± 1.57
Test	244	9670 ± 1270	8.93 ± 4.56	4.39 ± 1.49

approximately 2.25 and 2.36 million pulses over 245 and 244 scenarios, respectively. The emitter column counts the distinct emitters active in each scenario. Emitter identifiers are meaningful only within their own scenario (Section 4.1), so these figures describe how many emitters each scenario contains rather than membership in some global emitter set; at inference, the within-window clustering accordingly never has to separate more emitters than the enclosing scenario contains. The number of unique emitters per scenario ranges from 1 to 23 and the number of unique modes per scenario from 1 to 7 in every split, so the three partitions are statistically homogeneous in this respect. The global mode catalogue comprises $M = 7$ operating modes (Section 4.1), all of which are exercised in each split.

The numbers of emitters and assigned modes per scenario are sampled uniformly; however, the resulting pulse counts per emitter and mode are highly imbalanced due to variation in mode behavior. For instance, scanning modes typically generate far fewer pulses than lock-on modes, and some modes differ by more than an order of magnitude in their base PRI. As a result, the number of pulses (PDWs) per emitter or mode is not balanced either within a scenario or across the corpus. Because the encoder sees only windows of fixed length L (Section 4.2), rare combinations of emitter, mode, and interleaving pattern can be missing from individual windows or represented by only a handful of pulses, which limits what any single forward pass can evidence about those labels.

5.2.1 Qualitative properties of the synthetic corpus

Beyond aggregate counts, the corpus is checked qualitatively before large training runs: example timelines are inspected for plausible PRI structure per mode, for physically admissible RF and pulse-width ranges, and for the intended mixture density when multiple emitters are active. When the simulator regime adds dropped or spurious pulses (Section 4.2), spot checks confirm that surviving pulses retain their original time order and that spurious insertions stay within the configured parameter ranges. This mirrors the reporting practice in earlier Saab-hosted simulator studies, which often include illustrative emitter tracks alongside tables [14]. Section 5.6 complements the brief sanity checks here with embedding-space visualizations and other qualitative diagnostics once model checkpoints are available.

Table 5.2. Primary optimisation hyperparameters for the proposed model.

Setting	Value
Optimiser	Adam [20] with PyTorch defaults $(\beta_1, \beta_2) = (0.9, 0.999)$ and $\epsilon = 10^{-8}$
Peak learning rate $\eta_{\max}$	—
Minimum learning rate $\eta_{\min}$	0 (cosine floor; PyTorch default)
Cosine period $T_{\max}$	30 scheduler steps, one step per training epoch
Maximum epochs	30
Early stopping patience	7 epochs without improvement on the validation loss
Early stopping checkpoint	Weights from the epoch with lowest validation loss so far
Dropout (attention and feed-forward)	0.1 throughout the transformer blocks
Batch size B (windows per step)	—
Mode loss weight λ_{md}	—
Weight decay	—

5.3 Hyperparameters and Training Details

This section collects numerical training choices deferred from Sections 4.3, 4.5 and 4.6. Unless Section 5.5 states otherwise, baseline models use the same epoch budget, hardware (Section 5.1), and early-stopping protocol so that differences reflect architecture and objective rather than compute.

The contrastive temperature is fixed at $\tau = 0.2$ as in Section 4.5. Table 5.2 summarises the optimisation schedule and regularisation for the proposed dual-branch encoder; entries marked with an em dash are still being matched to the released training script and will be filled before the final thesis submission.

The cosine scheduler follows the PyTorch `CosineAnnealingLR` convention: after each epoch the learning rate is updated from $\eta_{\max}$ toward $\eta_{\min}$ over $T_{\max} = 30$ steps, which aligns the period with the nominal training horizon. If early stopping triggers first, training halts at the best validation checkpoint and the final few scheduler steps may not be executed.

5.4 Quantitative Results

This section reports primary clustering scores for the proposed model and for each baseline under the data conditions in Section 5.2. Rows index methods (including ablated variants where applicable); columns index emitter-level and mode-level AMI, ARI, and ACC after optimal alignment of cluster indices to the reference simulator labels (Section 4.7). Each entry is aggregated over random seeds and window splits as stated in Sections 5.1 and 5.3; mean ± one standard deviation is shown when multiple seeds are available.

5.5 Ablation Studies

Ablations isolate the contribution of individual components in Chapter 4 by retraining with exactly one change relative to the full configuration (matched data, seeds, and hyperparameters unless the ablation itself is a hyperparameter). Planned comparisons include removing or downweighting the emitter contrastive term, the mode contrastive term, or the coupling between branches; reducing the number of attention layers or heads; disabling stochastic augmentations beyond deterministic scaling; and swapping HDBSCAN post-processing for a simpler fixed-k assignment when that variant is implemented.

5.6 Qualitative Analysis

Qualitative analysis complements the scalar metrics in Section 5.4 by checking whether embeddings organize pulses in ways that align with physical emitter boundaries and with mode transitions inside a window. The checks in Section 5.2.1 validate that the simulator export behaves as intended; this section applies analogous scrutiny *after* models are trained.

Two-dimensional projections of pooled or per-pulse embeddings (for example UMAP or t-SNE) should be colored separately by reference emitter (using the train-local emitter label of each pulse) and by reference mode (using the global mode catalogue from Section 4.1). Disjoint emitter-colored manifolds with mode-colored substructure that stays coherent within an emitter are evidence that the dual geometry in Chapter 4 is behaving as designed; elongated bridges between emitter manifolds often indicate residual deinterleaving ambiguity or shared waveform parameters across platforms.

Attention maps or head-wise statistics, if exported, can be overlaid on TOA-ordered pulses to see whether specific heads track stable intervals versus rapid mode hops.

Confusion matrices between inferred clusters and reference labels (after optimal assignment) are optional but useful when a small number of emitter or mode classes dominate error mass.

CHAPTER 6

Discussion

6.1 Interpretation of Results

This section connects the quantitative tables in Section 5.4 and the qualitative material in Section 5.6 to the modeling choices in Chapter 4. The goal is not to restate individual scores, but to explain *which* mechanisms plausibly drive the observed ranking of methods and ablations.

When emitter-level AMI or ARI lag while mode-level scores are stronger, a natural hypothesis is that the encoder prioritizes intra-window structure tied to waveform parameters (which modes share) before it fully exploits slower cues that separate physical platforms across mixed traffic. Conversely, if emitter scores dominate but mode partitions collapse, the dual-head coupling or the mode contrastive term may be underweighted relative to the emitter branch, or certain modes may be too short-lived inside length-L windows to support stable clusters.

The dropped and spurious-pulse regimes produced by the simulator (Section 4.2) interact with self-attention in two ways: they perturb local timing patterns that define modes, and they change which pulses co-occur inside a window, which can confuse any window-level emitter summary. Interpretation should therefore read noise sweeps alongside the corpus diagnostics in Sections 5.2.1 and 5.6: a metric drop that coincides with visibly corrupted tracks is evidence of a robustness limit, whereas a drop on clean held-out scenarios points toward generalization or optimization issues instead.

6.2 Comparison with Related Work

Relative to classical ELINT fingerprinter pipelines surveyed in Section 3.1, the proposed approach trades hand-engineered feature logic for a learned encoder, at the cost of data hunger, compute, and reduced transparency unless accompanied by diagnostics such as those in Section 5.6. Unlike the supervised radar classifiers discussed in Section 3.2, which typically assume a fixed global label set for emitters and a fixed mode catalogue at inference, the training objective here treats emitter identity as a *train-local* symbol used only through within-train comparisons (Section 4.5); operating-mode labels are used globally during training to shape the mode embedding geometry, but the inference path runs through clustering rather than through a softmax over the training catalogue, so that the same model can accommodate modulation types absent from the training data.

Deep clustering methods in Section 3.3 typically optimise a *single* flat partition without labels, or attach two unrelated supervised heads to the same backbone. This thesis instead targets two aligned partitions with a nesting interpretation (modes within emitters), with two supervised contrastive losses that share the same functional form and differ only in the scope of their positive sets: within-window for emitter labels and batch-wide for global mode labels. The setup is closer in spirit to offline pulse-level grouping work that reasons about geometry and overlap in ESM data [25] but uses gradient-based representation learning rather than hand-tuned density stages alone. Where scores fall short of the best published supervised benchmarks, the gap is expected: those models often operate on curated single-emitter tracks with a flat, globally consistent emitter label set, whereas the present setting stresses mixture windows and a label structure that forces emitter inference through clustering rather than through a global classifier.

6.3 Limitations

The experimental programme is offline: models are trained and scored on bounded, revisit-able windows rather than under the latency, memory, and non-stationarity constraints of a continuously arriving pulse stream (Sections 1.4, 1.4.2 and 4.2). That setting supports controlled optimization and reproducible metrics but does not demonstrate that the same representations would support stable online emitter or mode tracking without periodic retraining or revised windowing policy.

Fixed-length windows are partly a consequence of full self-attention, whose standard implementation scales quadratically with the number of pulses in the window (Section 4.2). The chosen cap on L therefore trades faithful coverage of long mode segments and rare emitters against compute, and the present pipeline drops trailing segments shorter than L, which can remove evidence entirely when a scenario does not fill an integral number of windows.

Interleaved emitters and mode-hopping further skew how many pulses from each ground-truth label appear inside any given window, so performance summaries aggregate over heterogeneous label incidence rather than over an idealized balanced design (Section 5.2). Where metrics depend on comparing inferred partitions to reference labels, conclusions are most informative for label configurations that actually recur with sufficient mass inside windows; weak support for a label in a window is a limitation of the observation model, not necessarily of the encoder in isolation.

Finally, all quantitative evidence is derived from synthetic scenarios with full reference labels; operational recordings with imperfect deinterleaving, sensor dropouts, and distribution shift are not included here. Related Saab-line work on track association under deinterleaving errors highlights that real pipelines fragment pulse trains and introduce structured mistakes that are only partly captured by the stylized drop and spurious models used for stress tests [26]. Likewise, classifiers trained on simulator draws can exhibit overconfidence on held-out emitters or modulation families, which motivates explicit out-of-distribution and drift analyses when moving toward fielded receivers [27]. The present thesis does not implement drift monitors or uncertainty heads; those remain orthogonal safeguards if embeddings from Chapter 4 are ever deployed under evolving threat libraries.

The HDBSCAN post-processing used in Section 4.3 also encodes density assumptions that may not match every deployment, and we do not exhaust efficient-attention or hierarchical alternatives that could extend context within the same hardware envelope.

6.4 Implications

For ELINT and ESM practice, the main takeaway is methodological: joint emitter- and mode-aware embeddings can be learned from PDW streams whose supervision is structured asymmetrically (train-local emitter labels, global mode catalogue), with downstream discretisation handled by clustering on *both* branches so that previously unseen emitters and modulation types can in principle surface as distinct clusters rather than be silently absorbed into existing catalogue entries. Operators should treat the resulting per-pulse emitter and mode cluster indices as hypotheses that require human or library-based adjudication before tactical use, especially when training data are synthetic and drift relative to live emitters or to mode catalogues is unmodelled; in particular, mode-branch clusters whose count exceeds the size of the training catalogue, or that occupy regions of the mode embedding space far from those populated by training-time modes, are informative as candidate novel modulations and warrant separate review.

For the deep-clustering and sequence-modeling research communities, the thesis sits at the intersection of contrastive representation learning on irregular pulse sequences and multi-partition clustering with a physical nesting constraint (modes within emitters). The architectural and loss choices in Chapter 4 are portable to other domains with hierarchical latent structure, provided that evaluation metrics separate which level of the hierarchy is being scored.

Follow-up work naturally includes streaming or sliding-window evaluation, richer deinterleaving and track-error models [26], and explicit ML safeguards against dataset shift and overconfident extrapolation [27]. On the ethics side, the considerations in Section 1.6 still apply: dual-use sensitivity, data stewardship if operational traces are introduced later, and the energy cost of transformer-scale training should be revisited whenever the experimental footprint grows.

CHAPTER 7

Conclusion

7.1 Summary

Placeholder paragraph.

7.2 Answers to the Research Questions

Placeholder paragraph.

7.3 Future Work

Placeholder paragraph.

References

[1] R. G. Wiley, *ELINT: The Interception and Analysis of Radar Signals*. Artech House, 2006.

[2] M. I. Skolnik, *Radar Handbook*, 3rd ed. McGraw-Hill, 2008.

[3] Z. Qu, J. Zhang, Y. Zhou, and L. Ni, "The intelligent evolution of radar signal deinterleaving: A systematic review from foundational algorithms to cognitive AI frontiers", *Sensors*, vol. 26, no. 1, 2026. DOI: 10.3390/s26010248

[4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise", in *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.

[5] E. Gunn, A. Hosford, D. Mannion, J. Williams, V. Chhabra, and V. Nockles, *Radar pulse deinterleaving with transformer based deep metric learning*, arXiv preprint arXiv:2503.13476, 2025. arXiv: 2503.13476 [cs.LG].

[6] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, "Hierarchical density estimates for data clustering, visualization, and outlier detection", *ACM Transactions on Knowledge Discovery from Data*, vol. 10, no. 1, 2015. DOI: 10.1145/2733381

[7] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering", *The Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017. DOI: 10.21105/joss.00205

[8] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis", in *International Conference on Machine Learning (ICML)*, 2016, pp. 478–487.

[9] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations", in *International Conference on Machine Learning (ICML)*, 2020.

[10] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, arXiv preprint arXiv:1807.03748, 2018. arXiv: 1807.03748 [cs.LG].

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding", in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[13] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, "Unsupervised learning of visual features by contrasting cluster assignments", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[14] T. Jonsson, "Classification of Radar Emitters using Semi-Supervised Contrastive Learning", Host company Saab AB; supervised learning experiments on pulse data from the OpenModesty scenario simulator, Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2023.

[15] P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations", in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2018, pp. 464–468. DOI: `10.18653/v1/N18-2074`

[16] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, "Do transformers really perform bad for graph representation?", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 28 877–28 888.

[17] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding", *Neurocomputing*, vol. 568, p. 127 063, 2024. DOI: `10.1016/j.neucom.2023.127063`

[18] B. Heo, S. Park, D. Han, and S. Yun, "Rotary position embedding for vision transformer", in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024, pp. 289–305. DOI: `10.1007/978-3-031-72684-2_17`

[19] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[20] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, arXiv preprint arXiv:1412.6980, 2017. arXiv: `1412.6980 [cs.LG]`.

[21] L. Hubert and P. Arabie, "Comparing partitions", *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[22] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance", *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.

[23] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure", in *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 2007, pp. 410–420.

[24] H. W. Kuhn, "The Hungarian method for the assignment problem", *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955. DOI: `10.1002/nav.3800020109`

[25] S. Bedoire, “Offline direction clustering of overlapping radar pulses from homogeneous emitters”, Host company Saab; DBSCAN-style clustering on pulse directions with simulated scenarios, Master’s thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2022.

[26] V. Jakobsson, “Offline radar pulse track association with deinterleaving errors”, Host company SAAB; time-series clustering for track fragments under imperfect deinterleaving, Master’s thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2024.

[27] K. Coleman, “Dataset drift in radar warning receivers: Out-of-distribution detection for radar emitter classification using an RNN-based deep ensemble”, UPTEC F 23041, 30 hp; simulator data from Saab AB and drift/OOD analysis for emitter classification, Master’s thesis, Uppsala University, Uppsala, Sweden, 2023.

APPENDIX A

Additional Results

Placeholder paragraph.

APPENDIX B

Implementation Details

Placeholder paragraph.

APPENDIX C

Notation Reference

Placeholder paragraph.